UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

PROIECT DE DIPLOMĂ

Cristian-Gabriel SPINU

COORDONATOR ȘTIINȚIFIC
Conf. dr. ing. Ileana-Diana NICOLAE

CRAIOVA, ROMÂNIA
Septembrie 2026

UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

Advanced Software for Human Resources Management

Cristian-Gabriel SPINU

COORDONATOR ȘTIINȚIFIC

Conf. dr. ing. Ileana-Diana NICOLAE

CRAIOVA, ROMÂNIA

Septembrie 2026

ABSTRACT

This thesis presents the design and implementation of a web-based CRM (Customer/Candidate Relationship Management) application for managing an organization's projects, its pool of human resources, and the assignment of people to projects on the basis of skill fit. The motivation is a concrete operational problem: coordinating projects and staff availability through generic tools (spreadsheets, disconnected documents) becomes unmanageable as the number of projects and people grows, and the core staffing question — "who best fits this project?" and, symmetrically, "which open project best fits this person right now?" — remains a manual, unaudited decision.

The solution is a full-stack application with a strict separation of concerns: a Go backend exposing a schema-first GraphQL contract (via gqlgen) over a PostgreSQL database, and a React frontend that consumes that contract exclusively, using Apollo Client as its server-state cache. The functional core of the application is a symmetric skill-matching scoring algorithm, implemented once at the service layer and exposed in two complementary directions: project-to-resource (ranking candidate resources for a project's open positions) and resource-to-project (ranking open projects a given resource best fits). Around this core, the application covers full project and resource management, a shared skill catalog, CV generation from configurable HTML templates, JWT session authentication, per-record author attribution, and a reporting dashboard with aggregate KPIs (staffing fill rate, skill demand vs. supply, resource-status distribution).

The thesis documents both the architectural decisions (why GraphQL as the contact layer, why a single symmetric matching service instead of two separate implementations, why TEXT with CHECK constraints instead of native PostgreSQL enums for fields expected to evolve) and the practical outcome: a fully functional, Docker-containerized system with versioned database migrations and an automated test suite covering the dependency-free parts of the business logic (the matching score, report-summary computation, CV template validation).

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

1 INTRODUCTION

1.1 Purpose of the Thesis

The purpose of this thesis is to design and implement an internal CRM-type web application that covers, within a single coherent system, three needs of an organization that delivers projects using its own staff:

1. **tracking ongoing projects**, with their status (planning, active, on hold, completed) and their staffing requirements expressed per skill (for example, "2 Go developers, 1 UI designer");
2. **tracking the available human resources** — each person's skills, availability over time, contact details, activity history and known languages — including the ability to generate a CV automatically from a template;
3. **allocating staff to projects** on the basis of the fit between required and held skills, with decision support (a matching score) in both directions: starting from a project ("who fits?") and starting from a person ("what is this person a fit for right now?").

The intended outcome is not merely a database with a user interface, but a tool that reduces the time and effort required for an allocation decision, by aggregating the relevant information (skills, availability, current workload) in a single place and by computing a matching score automatically, instead of manually checking a list of people row by row.

1.2 Motivation for Choosing the Topic

The motivation for the project is twofold. On the one hand, skill-based project staffing is a real and frequent problem in service organizations (IT consultancies, agencies, internal delivery teams), usually solved ad hoc with manually maintained spreadsheets, without a change history and without any automatic recommendation mechanism. On the other hand, the topic offers suitable ground for practising, in a realistically sized project, a set of modern full-stack software engineering practices: *schema-first* design of the client-server contract (GraphQL), strict separation of the GraphQL resolvers from the business logic (a service layer), versioned management of the database schema (migrations), full containerization of the solution, and writing automated tests for the pure business logic, independently of infrastructure.

The choice of technology stack — **React** with **Apollo Client** on the frontend, **Go** with **gqlgen** on the backend, **GraphQL** as the contract language and **PostgreSQL** as the database — reflects an explicit preference for open-source tools that are typed wherever possible (Go on the backend, types generated

from the GraphQL schema) and for pragmatism: each technology was chosen for the problem it solves directly, without additional layers of abstraction on top of what GraphQL or Apollo Client already provide natively.

1.3 Current Context

The market of tools for project and resource management is dominated by generalist solutions (Jira, Monday.com, Asana) or by large-scale ERP/HR systems, both of which are either too generic to model the "required skill ↔ held skill" relationship directly, or too complex and costly for a small or medium team that needs only this functional core. The current technological context — the maturity of GraphQL as an alternative to REST for typed client-server contracts, code generation from the schema (both on the backend with `gqlgen` and, potentially, on the frontend), and the availability of easily installed containerized tooling (Docker, `docker-compose`) — makes it feasible to build, within a reasonable time, such a dedicated tool, tailored exactly to the workflow described above, without paying the complexity cost of a generalist system.

2 SYSTEM ANALYSIS AND SOLUTION ARCHITECTURE

2.1 Functional and Non-Functional Requirements

The functional requirements of the system are derived directly from the use cases documented in full in Chapter 4 and can be grouped as follows:

- **Authentication and session**: authentication based on username/e-mail and password, a session with a limited lifetime (a signed token, valid for 24 hours), explicit logout, automatic redirection to the login screen for any protected route accessed without a valid session.
- **Project management**: creation, deletion, status change, tagging with skills, definition of staffing requirements per skill (with tracking of the fill level), comments.
- **Resource management**: creation, editing, deletion, tagging with skills, blocking (blacklisting) with automatic release of current assignments, comments, activity history (work periods), known languages with a proficiency level (CEFR), CV generation.
- **Skill catalog**: a single, shared catalog, used for tagging both projects and resources.
- **CV templates**: definition, editing, deletion and marking of a default (HTML) template, with structural validation before saving.
- **Staff allocation (matching)**: recommendation of candidates for a project (with filtering by availability, skill and text search) and, symmetrically, recommendation of open projects for a given resource; actual assignment and unassignment.
- **Reports**: aggregate indicators (KPIs), a required-vs-assigned chart per project, a skill demand-vs-supply chart, and the distribution of resources by status and availability.

The non-functional requirements pursued were:

- **End-to-end typing**: the GraphQL schema is the single source of truth for the shape of the data; the Go types and the frontend TypeScript/JavaScript types derive from it, not the other way around.
- **Traceability**: every record created or modified retains the identity of the authenticated user who made the change, rather than a generic system account (section 3.1.4).
- **Data consistency**: operations that touch several tables at once (for example, marking a CV template as the default, which must automatically clear the previous default) run within a single transaction.
- **Deployment portability**: the system must be startable with a single command (`docker compose up`), with no manual database configuration steps.

- **Testability**: the business logic with decision value (the matching score, the computation of report indicators) must be testable without a real database.

2.2 Overall Architecture

The application follows a classic three-tier client-server architecture, communicating exclusively through typed contracts, as shown in **Figure 2.1**: the React client speaks GraphQL over HTTP/JSON to the Go backend, which in turn reaches PostgreSQL through `sqlx`. The figure also shows the three Docker services the solution runs from, together with the optional `seed` profile.

The frontend never accesses the database or any service other than the backend's single GraphQL endpoint (`/graphql`); the backend, in turn, exposes no auxiliary REST endpoint for application data — the only additional HTTP routes are the GraphQL endpoint itself and a development playground. This single-contract discipline is what makes automatic type generation on both sides from the same `.graphql` schema possible, and avoids the type drift that frequently appears when the client and the server define the shape of the same data separately and by hand.

On the backend, the layers are explicitly separated:

- **GraphQL resolvers** (`backend/graph/*.resolvers.go`, partially generated by `gqlgen`) — thin, delegating immediately to the service layer; they contain no business logic.
- **Service layer** (`backend/internal/service/*.go`) — one service per domain (`ProjectService`, `ResourceService`, `SkillService`, `MatchService`, `ReportService`, `CvService`, `AuthService`, `CommentService`, `ResourceActivityService`, `LanguageService`), each encapsulating the SQL queries and business logic of its own domain.
- **Data access** — directly via `sqlx` over the `lib/pq` driver, without an ORM; the queries are plain, explicitly written SQL, never `SELECT *`.

Figure 2.2 shows this layering, together with the point at which each cross-cutting concern (CORS, the authentication middleware, the `@auth` directive) intervenes.

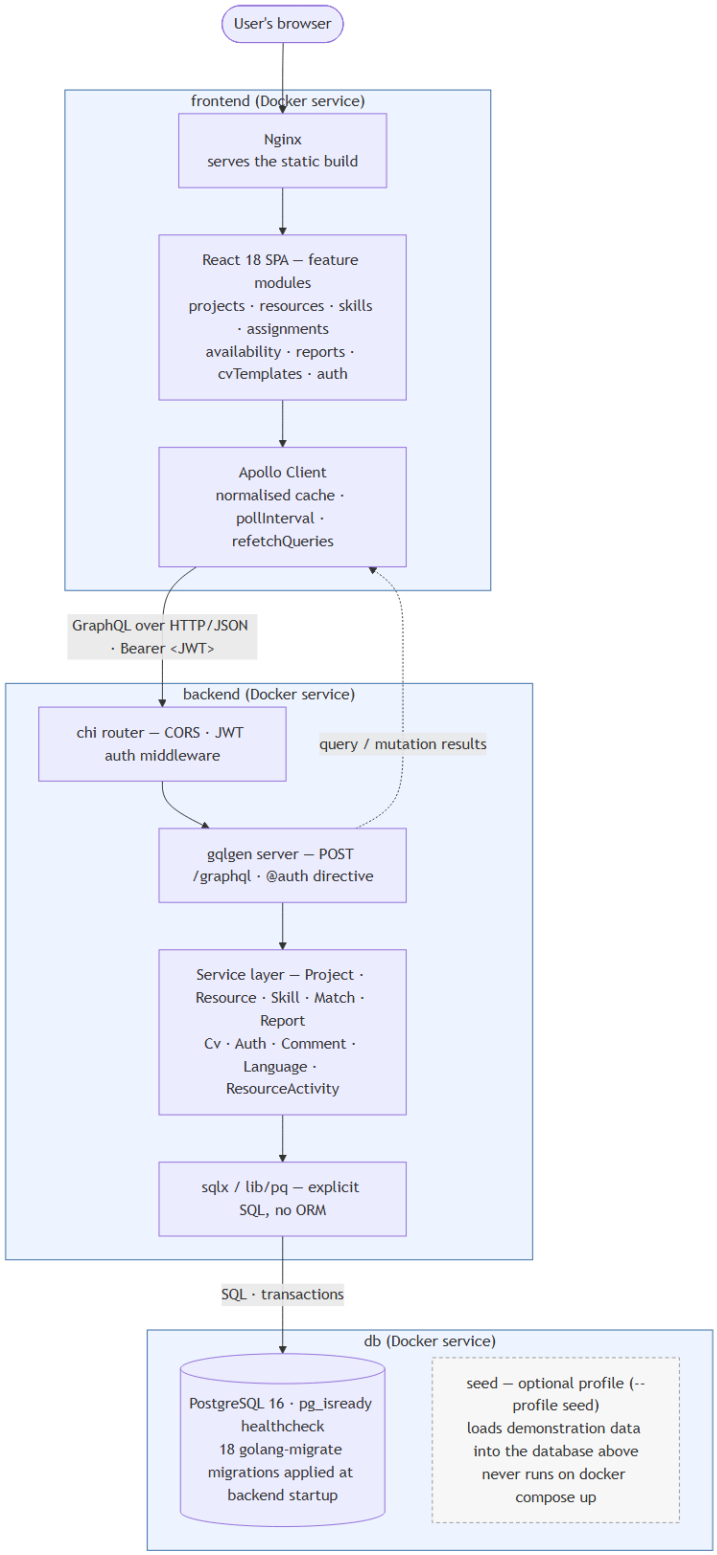

Figura 2.1. Overall architecture: client, server and database.

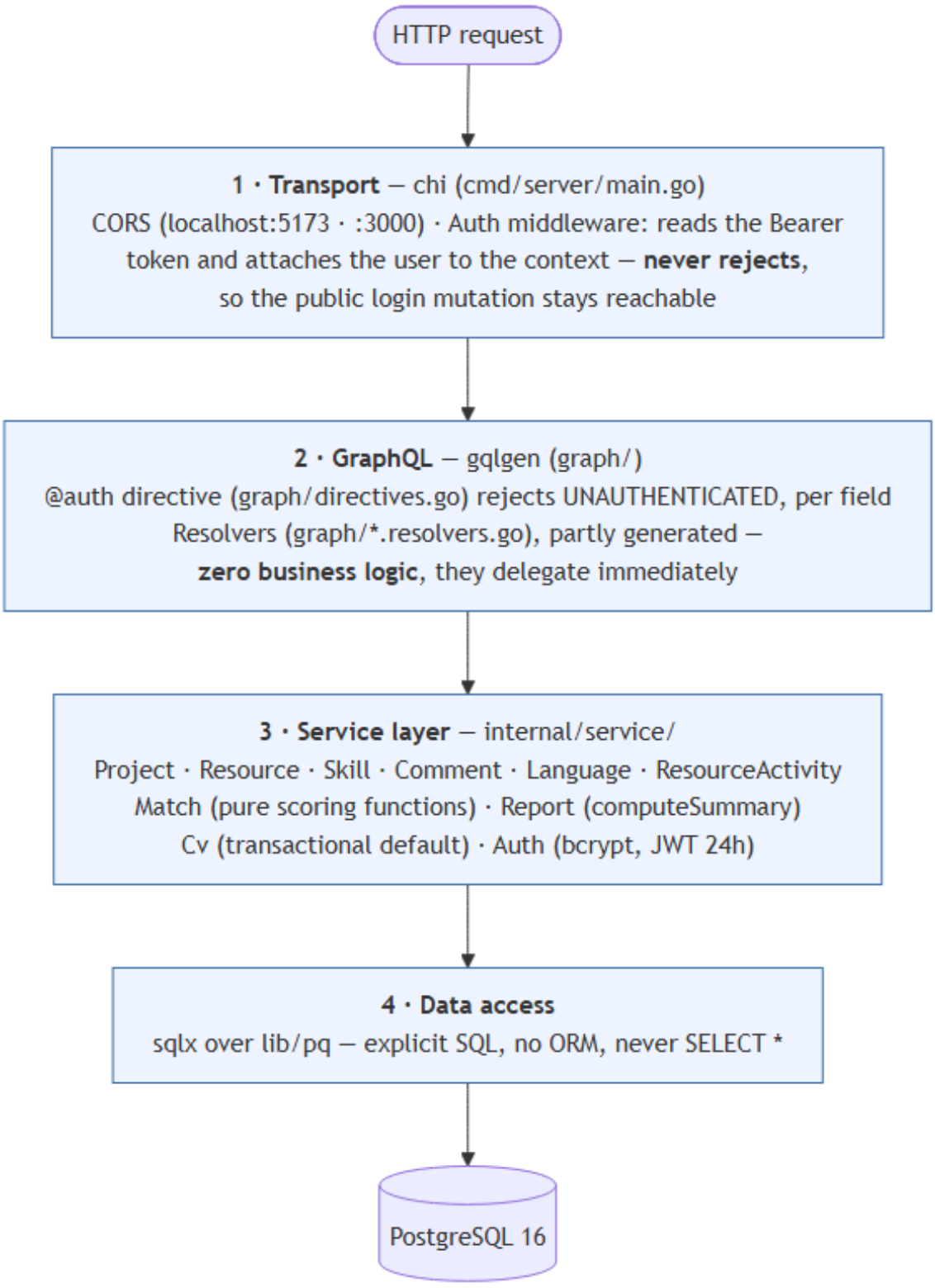

Figura 2.2. Backend layering: thin resolvers over a service layer.

2.3 The Data Model

The database schema is managed through versioned migrations (`golang-migrate`, 18 `up`/`down` file pairs in `backend/migrations/`), applied automatically when the server starts. The main entities are:

Tabel 2.1. The main database entities.

Entity	Role
`projects`	A project: name, contact person, phone, e-mail, status, author and last modifier
`resources`	An available person: contact details, full address, driving licence, own car, declared availability, status (`FREE` / `ASSIGNED_TO_PROJECT` / `BLACKLIST`)
`skills`	The single, shared skill catalog (unique name, optional description)
`project_skills` / `resource_skills`	N-to-N association tables between projects/resources and skills (tagging)
`project_skill_requirements`	A project's staffing requirements, per skill (`needed`, with `filled` derived from the current assignments)
`project_assignments`	The actual assignment of a resource to a project, optionally linked to a specific required skill
`project_comments` / `resource_comments`	Comment threads, attributed to their author and timestamped
`resource_activities`	A resource's activity history (work periods, with a start/end date)
`languages` / `resource_languages`	A catalog of languages and each resource's proficiency level (CEFR A1–C2, or Native)
`cv_templates`	HTML templates for CV generation, with at most one template marked as default (enforced by a partial unique index)
`users`	The application accounts (username, e-mail, password hash, display name)

The complete entity-relationship diagram is split across two figures, for legibility: **Figure 2.3a** shows the entities that participate directly in matching (projects, resources, skills, the association tables, the staffing requirements and the assignments), while **Figure 2.3b** shows the supporting entities (comments, activity history, languages, CV templates and accounts).

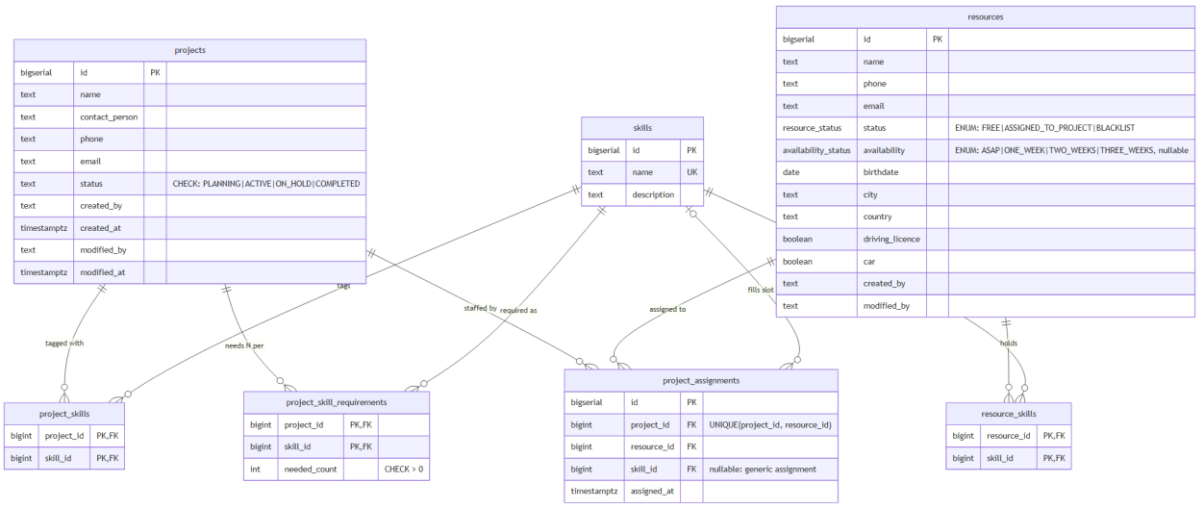

Figura 2.3a. Entity-relationship diagram: the matching core.

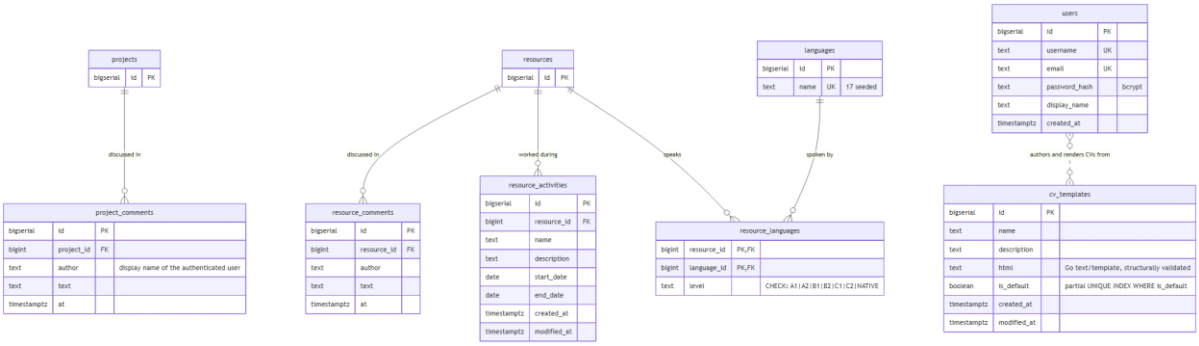

Figura 2.3b. Entity-relationship diagram: the supporting entities.

A deliberate design decision, applied consistently throughout the schema, is the preference for `TEXT` columns with `CHECK` constraints (for example, the language level in `resource_languages`) instead of native PostgreSQL `ENUM` types, wherever the set of values may evolve: in PostgreSQL, a label of an `ENUM` type cannot be renamed or removed without recreating the type, which in practice genuinely complicated the evolution of the `resource_status` field during development (migrations 000007–000008 in the schema history document exactly this correction). Fields that are not expected to evolve frequently (a resource's status, its availability) nevertheless remained native `ENUM`s, where the database-level integrity benefit outweighs the cost of rigidity.

2.4 The GraphQL Contract

The GraphQL schema is split into modules, one `.graphql` file per domain (`backend/graph/schema/`): `auth`, `project`, `resource`, `skill`, `comment`, `cv`, `language`, `resource_activity`, `report`. The conventions observed throughout the schema are:

- types in `PascalCase`, fields in `camelCase`, mutations named as verbs (`createProject`, `assignResource`, `removeSkill`);
- all arguments of a mutation are grouped into a dedicated `Input` type, rather than passed as individual parameters;
- fields are non-null (`!`) by default; a field is nullable only when its absence carries real meaning (for example, `availability` on a resource may be unknown);
- every field or operation that requires a valid session is explicitly marked with the custom `@auth` directive, applied at field level — authorization is not an implicit assumption, but an annotation visible directly in the schema.

Tabel 2.2. The GraphQL schema modules.

Module	Main content
`auth.graphql`	`login` (a public mutation, the only one without `@auth`)
`project.graphql`	The `Project` type, CRUD, statuses, staffing requirements, `matchResources`
`resource.graphql`	The `Resource` type, CRUD, blacklisting, `matchProjects`, `resourceAllocations`
`skill.graphql`	The skill catalog
`cv.graphql`	CV templates and document generation
`language.graphql`	The language catalog and per-resource levels
`resource_activity.graphql`	A resource's activity history
`comment.graphql`	Comments on projects and resources
`report.graphql`	The aggregate dashboard indicators

2.5 Authentication and Traceability

Authentication uses a signed token (JWT), issued at login and validated on every subsequent request by an HTTP middleware at router level (`chi`). The authentication middleware has a strictly limited role — it reads the `Authorization: Bearer …` header, validates the signature and, if valid, attaches the user's identity to the request context; it does not, however, reject requests without a token, because the same `/graphql` endpoint must remain accessible for the `login` mutation, which is public by definition. Authorization is actually enforced at field level, through the `@auth` directive, checked during the execution of each GraphQL request.

The password is stored exclusively as a `bcrypt` hash; verification at login compares the hash, not the plaintext password. The failed-authentication error is single and generic ("invalid username/email or password"), regardless of whether the real reason was a non-existent account or a wrong password — a deliberate practice of not confirming to an attacker the existence of an account through the error message.

Author traceability is implemented as a cross-cutting concern: every creation or modification of a project, a resource or a comment reads the current user's display name from the session and writes it into the `createdBy`/`modifiedBy` field (or as the comment's author), instead of a fixed system-account value. This behaviour is documented as a standalone cross-cutting use case (UC SUP-2, Chapter 4), because it participates in nine different flows without having a screen of its own.

Figure 2.4 follows a single request end to end — from the `useQuery` call in a React component, through the Apollo cache, the authentication middleware and the `@auth` directive, down to the SQL query and back — making visible the division of responsibility described above: the middleware only *attaches* an identity, while the directive is what actually *enforces* authorization.

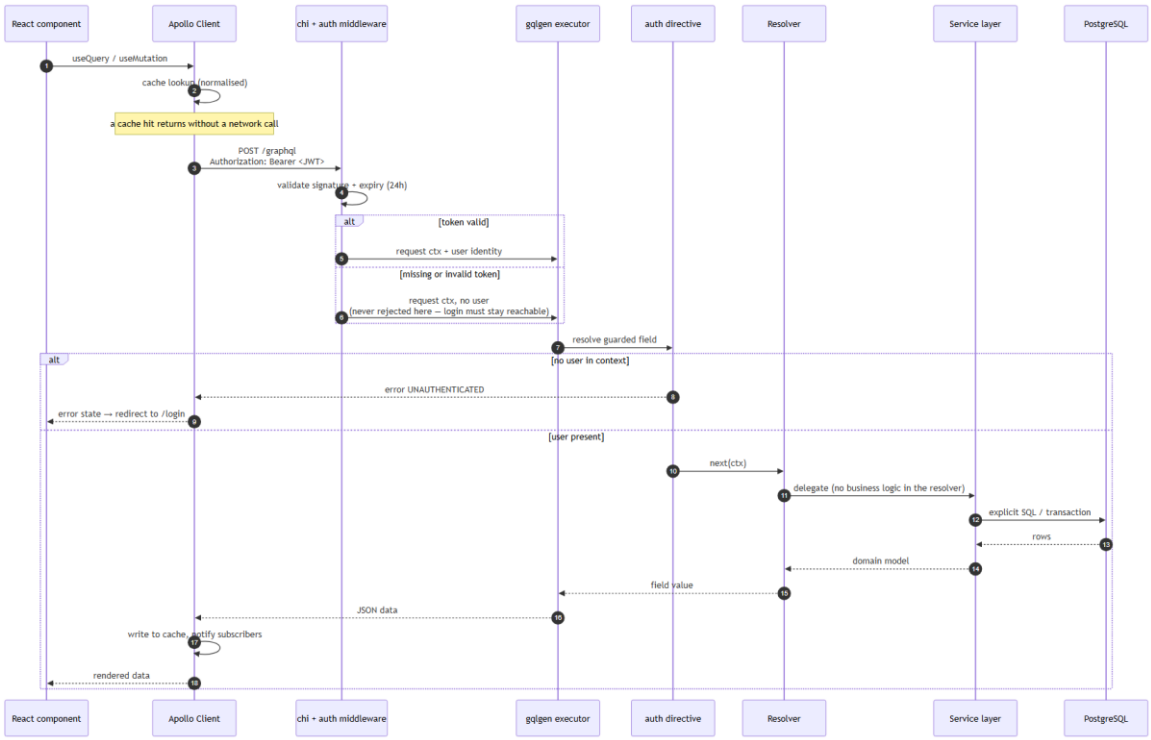

Figura 2.4. The lifecycle of a GraphQL request, from the React component to the database.

2.6 Containerization and Deployment

The entire solution runs from three Docker services orchestrated through `docker-compose.yml`: `db` (PostgreSQL 16, with a `pg_isready` health check), `backend` (an image built from a multi-stage `Dockerfile` in `backend/`, which applies the migrations automatically at startup) and `frontend` (a static React build served through Nginx). A fourth service, `seed`, is defined under an optional profile (`--profile seed`) and populates the database with demonstration data, never running implicitly on `docker compose up` — a deliberate decision, so that a production environment does not accidentally receive test data.

3 IMPLEMENTATION

3.1 Backend (Go + gqlgen)

The backend is structured into the three layers described in section 2.2. A representative example of the "thin resolver" discipline enforced throughout the code is the resolver for finding the projects that suit a resource:

```
func (r *queryResolver) MatchProjects(
    ctx context.Context, resourceID string, filter *model.ProjectMatchFilter,
) ([]*model.ProjectMatch, error) {
    matches, err := r.MatchSvc.MatchProjects(ctx, r.ProjectSvc, r.ResourceSvc,
resourceID, filter)
    if err != nil {
        return nil, gqlerror.Errorf("failed to match projects")
    }
    return matches, nil
}
```

The resolver contains no business decision — it delegates immediately to `MatchService` and translates the internal error into a structured GraphQL error (`gqlerror.Errorf`), without leaking implementation details (for example, a raw SQL message) to the client. The same convention is observed by all the other resolvers of the application.

Internal errors are wrapped consistently with `fmt.Errorf("%w", err)`, preserving the causal chain for debugging on the server, while the client always receives a controlled message, through `gqlerror.Errorf`.

3.1.1 The Matching Algorithm

The functional core of the application — the match between required and held skills — is implemented only once, in `MatchService`, in the form of pure functions that are testable independently of the database:

```
// scoreCandidate counts how many of the wanted skills a resource covers, and
// returns those overlapping skill ids in the order they were wanted.
func scoreCandidate(resourceSkillIDs, wantedSkillIDs []string) (int, []string) {
    have := make(map[string]bool, len(resourceSkillIDs))
    for _, id := range resourceSkillIDs {
        have[id] = true
    }
    matching := make([]string, 0, len(wantedSkillIDs))
    for _, want := range wantedSkillIDs {
        if have[want] {
            matching = append(matching, want)
        }
    }
    return len(matching), matching
}

// unfilledSkillIDs returns the skills a project still needs people for,
// falling back to its tagged skills when no per-skill requirements exist.
func unfilledSkillIDs(requirements []*model.SkillRequirement, skillIDs []string)
[]string {
    out := make([]string, 0, len(requirements))
    for _, r := range requirements {
        if r.Filled < r.Needed {
            out = append(out, r.SkillID)
        }
    }
    if len(requirements) == 0 {
        return skillIDs
    }
    return out
}
```

Two public methods of the same service are built on top of these two pure functions:

- **`Match(ctx, projectID, filter)`** — starting from a project, it computes the skills that are still uncovered (`unfilledSkillIDs`), then scores every eligible resource (`scoreCandidate`) and orders the result by score descending, then by availability (the nearest in time wins), then alphabetically by name.
- **`MatchProjects(ctx, resourceID, filter)`** — the mirror image: starting from a resource, it excludes completed projects and those the resource is already assigned to, computes the uncovered skills of each remaining project and scores the given resource against them, ordering by score descending, then by the number of still-open positions, then alphabetically by project name.

The fact that both directions of the matching reuse exactly the same two pure functions eliminates the risk of the two flows diverging over time (for example, one considering a skill "covered" differently from the other) — a divergence that would be easy to introduce if each direction computed its score independently. **Figure 3.1** makes this symmetry explicit: the two directions differ only in what they load and exclude beforehand, and in the criteria by which they break a tie afterwards, while the scoring itself passes through the same shared core.

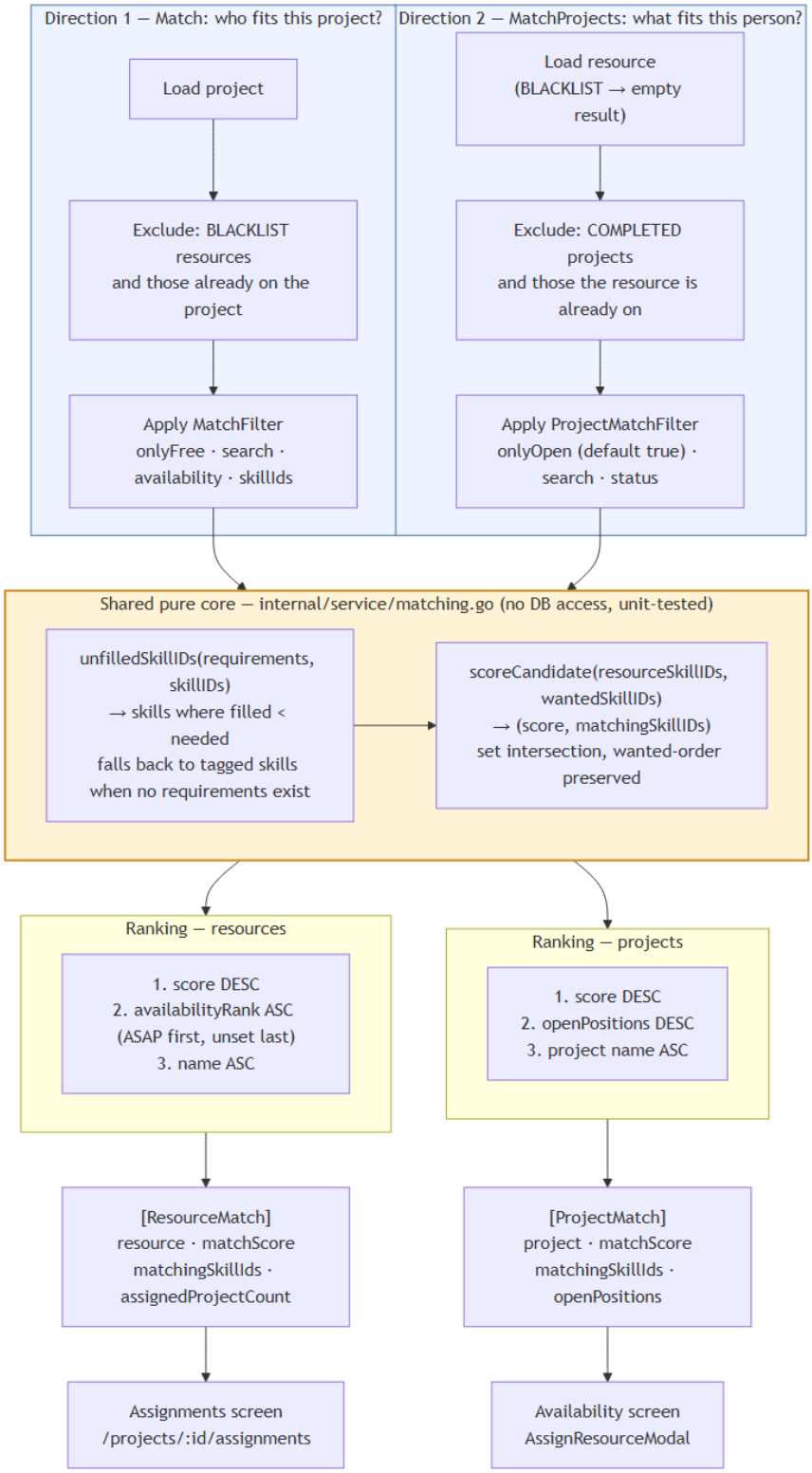

Figura 3.1. The symmetric matching algorithm: two directions over one shared pure core.

3.1.2 Transactional Consistency

Certain operations touch several rows at once and must be atomic. The most relevant example is marking a CV template as the default: the system guarantees, at application level, the invariant "at most one default template at any moment". In `CvService.CreateTemplate/UpdateTemplate`, if the new template is marked as default, the service opens a transaction, first executes `UPDATE cv_templates SET is_default = FALSE WHERE is_default`, then inserts/updates the new template with `is_default = TRUE`, and commits the transaction — so that no external observer (another concurrent request) can see, not even transiently, two templates simultaneously marked as default. The invariant is in fact doubly enforced, at database level as well, through a partial unique index (`CREATE UNIQUE INDEX … ON cv_templates (is_default) WHERE is_default`), so that a possible application error could never persist a violation of the rule.

3.1.3 Validation

The HTML content of a CV template is structurally validated before being saved (`validateTemplateHTML`), so that a broken template can never end up being used for generating a document — the error is rejected immediately, on save, with the editor remaining open for correction (UC CV-2/CV-3, Exception Flow E1).

3.1.4 Testing

The backend's automated tests are organized as *table-driven* tests, with no dependency on a real database, covering exactly those pure functions described above:

Tabel 3.2. The automated test suite (backend).

File	What it tests
`matching_test.go`	`scoreCandidate`, `unfilledSkillIDs`, the matching filters, in both directions
`report_test.go`	`computeSummary` — the edge cases (zero projects, zero requirements, over-allocation)
`cv_test.go`	Validation of the HTML structure of a CV template

The choice to test the business logic through pure functions, separated from data access, is what makes it possible to run this suite without starting a PostgreSQL instance — the tests run in milliseconds, as part of any `go test ./...`.

3.2 Frontend (React + Apollo Client)

The frontend is organized into feature modules (`frontend/src/features/{auth,projects,resources,skills,cvTemplates,assignments,ava`

ilability,reports}), each module grouping its own components, hooks and the `.gql.js` document with the GraphQL queries/mutations used by that module. All components are functional, using hooks; server data is fetched exclusively through `useQuery`/`useMutation` from Apollo Client — never through `useEffect` combined with manual calls.

Tabel 3.1. Resource statuses and their meaning.

Status	Meaning
`FREE`	The resource has no active assignment; it can be proposed for any open project
`ASSIGNED_TO_PROJECT`	The resource has at least one active assignment on a project
`BLACKLIST`	The resource is excluded from the allocation process; all its current assignments were removed on blocking

3.2.1 The Availability Screen and Resource-Side Assignment

A concrete example of how the frontend consumes the GraphQL contract is the **Availability** screen (`/availability`), which offers a resource-to-project view — complementary to a project's classic **Assignments** screen (`/projects/:id/assignments`), which looks from the project towards the resources (**Figure 3.2**).

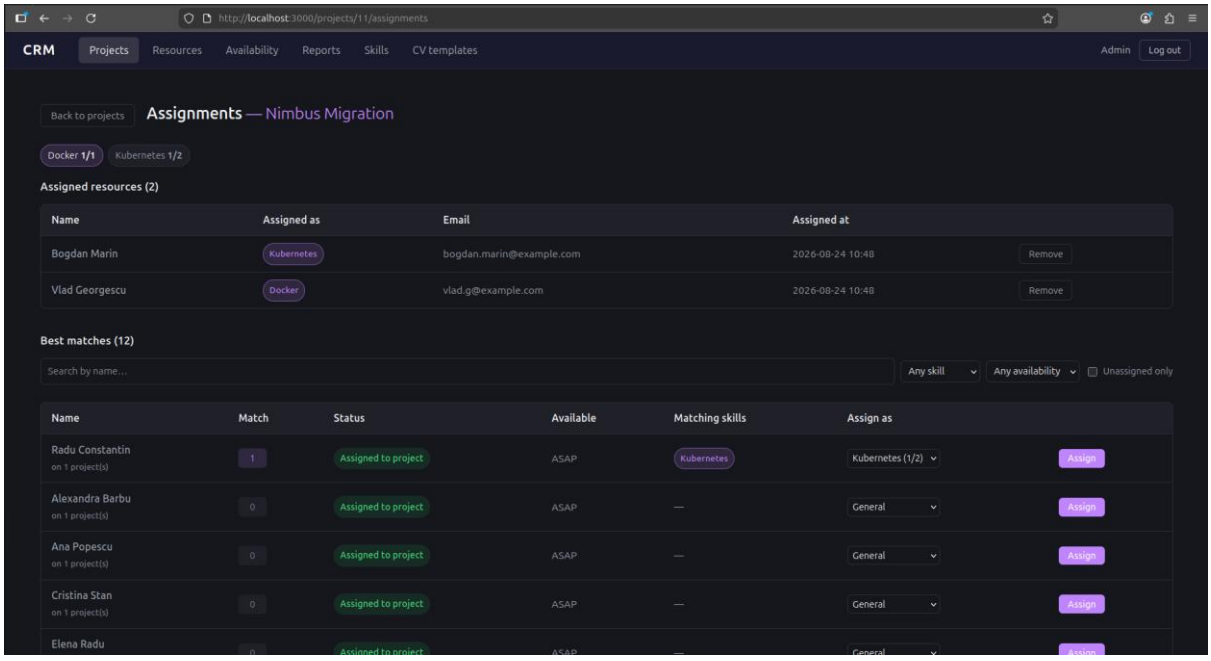

Figura 3.2. The Assignments screen, with candidate resources ranked by score.

(Screenshot to be captured — see `docs/figures/README.md`.) The `Availability` component queries `resourceAllocations` with automatic refresh every 15 seconds (`pollInterval`), so that the

screen stays current even if the assignments are modified from another session, with no action on the user's part:

```
const { data, loading, error } = useQuery(RESOURCE_ALLOCATIONS_QUERY, {
  pollInterval: POLL_INTERVAL_MS, // 15000
})
```

Selecting the "Assign" button on a resource opens the `AssignResourceModal` dialog, which queries `matchProjects(resourceId)` — the GraphQL endpoint described in section 3.1.1 — and displays the open projects ordered by score, with a selector for the specific position (skill) the assignment is made against, automatically pre-filled with the first uncovered requirement that the resource satisfies:

```
function slotFor(match) {
  if (slotChoices[match.project.id] !== undefined) return slotChoices[match.project.id]
  const open = (match.project.requirements ?? []).find(
    r => r.filled < r.needed && match.matchingSkillIds.includes(r.skillId),
  )
  return open?.skillId ?? ''
}
```

On confirmation, the `assignResource` mutation is sent with a list of `refetchQueries` covering all the screens affected by the result — the assignment list, the project list, the resource list and the matching result for the same resource — so that the effect of the assignment is visible immediately, without a manual page reload, on whichever screen displays it. **Figure 3.3** shows the screen with this dialog open, and **Figure 3.4** traces the complete flow of the operation, including the refresh cascade triggered on confirmation.

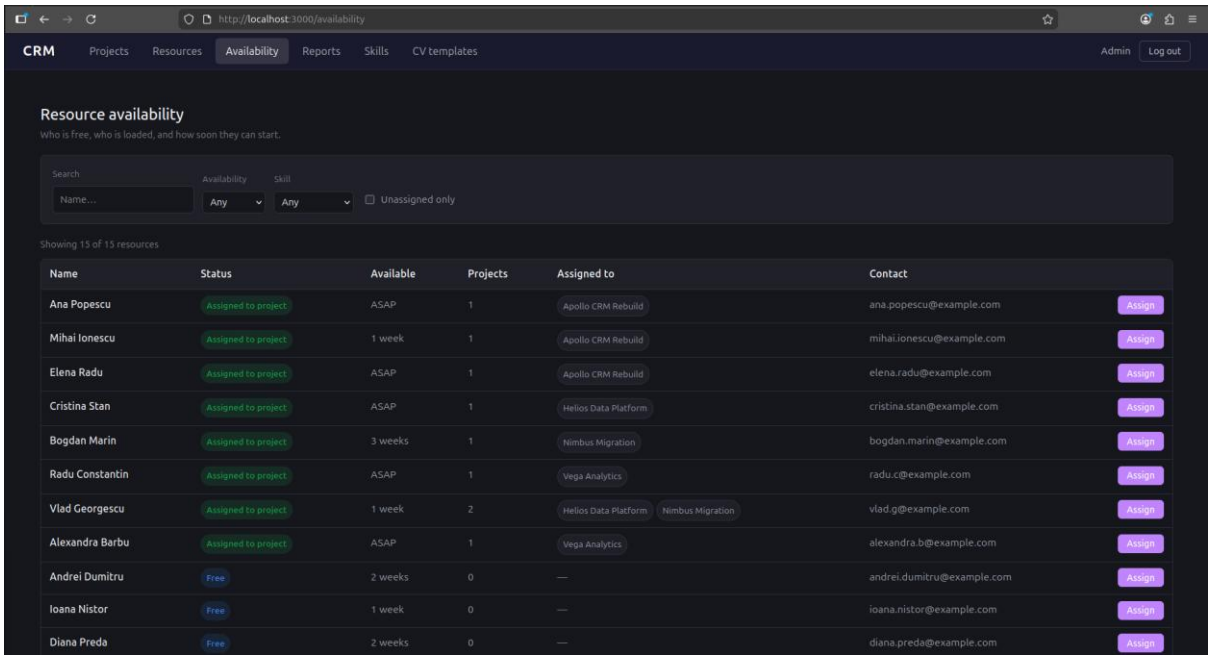

Figura 3.3. The Availability screen, with the resource assignment dialog.

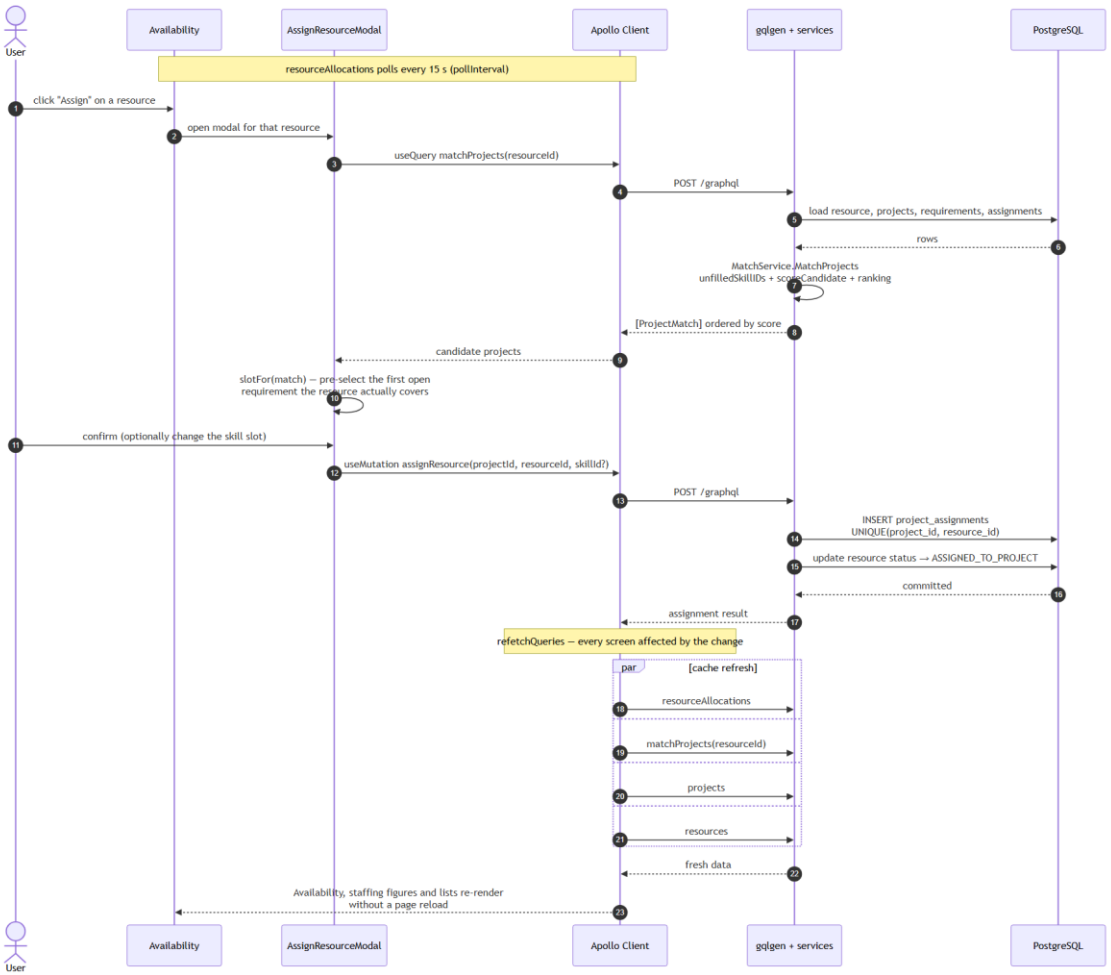

Figura 3.4. Assigning a resource to a project from the Availability screen.

3.2.2 Sortable Tables

The Projects and Resources lists use a generic column-sorting mechanism that is purely local (without an additional request to the server): selecting a column header sorts descending by that column; selecting the same header again reverses the direction. The behaviour is documented only once, as UC SUP-1 (Chapter 4), because it applies identically on both screens, avoiding duplication of the specification. **Figure 3.5** shows the project list with this sorting applied.

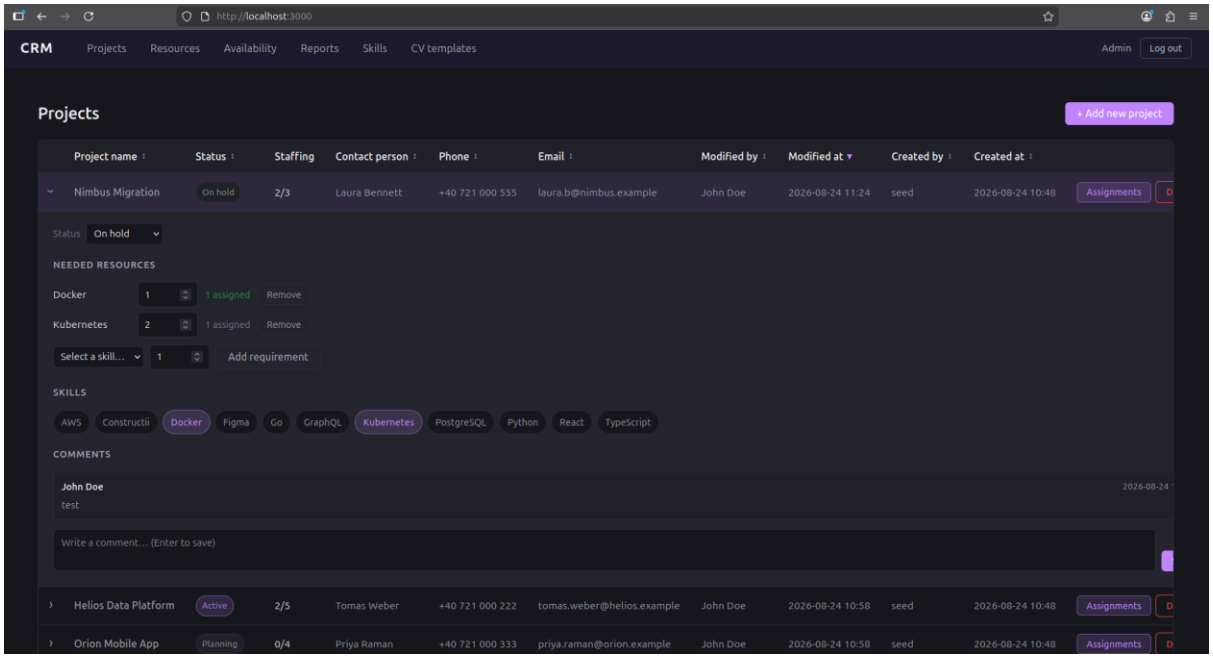

Figura 3.5. The project list, with sorting and an expanded detail panel.

3.2.3 CV Generation

From a resource's profile, a user can generate a CV starting from any defined template (Chapter 4, UC RES-10): the resource's data (skills, languages, activity history) is rendered into the chosen HTML template, and the result can be exported either as a PDF (through the browser's print dialog) or directly as an editable `.docx` document, generated on the server (`backend/internal/docx`). The resource's comments, status and current assignments are never included in the CV, regardless of the template — the CV is strictly a document oriented towards the person's professional profile. **Figure 3.6** shows this generation step, with the template selected and the result previewed.

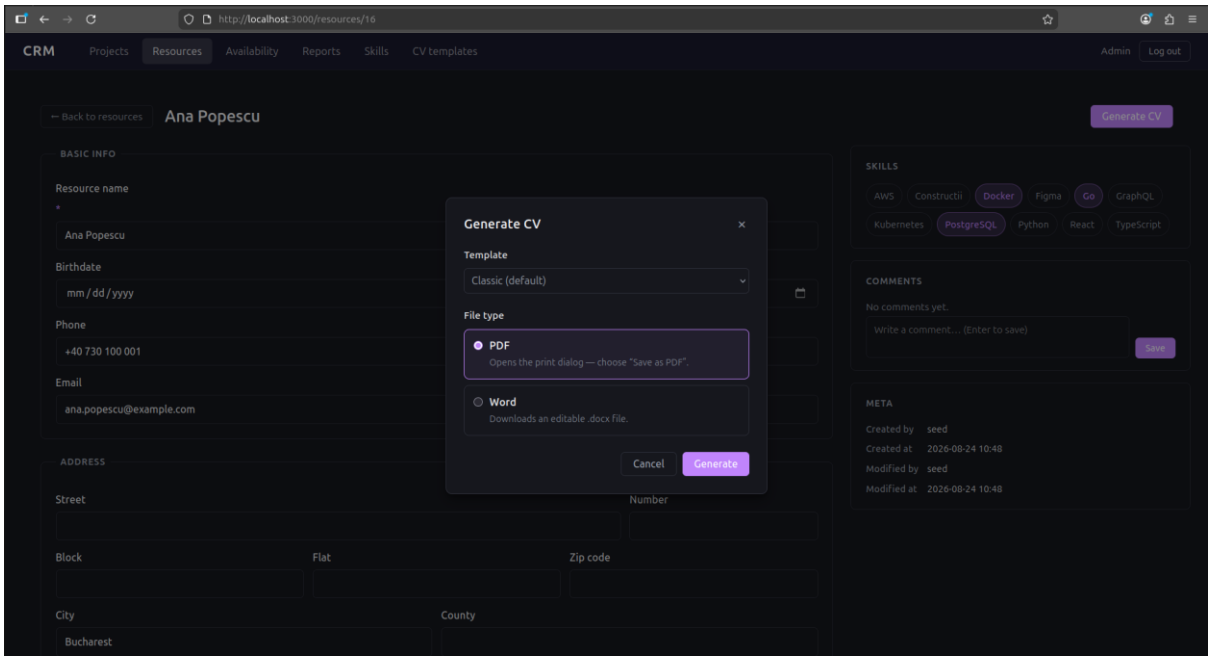

Figura 3.6. Generating a CV from a template.

3.2.4 The Reports Dashboard

The **Reports** screen (`/reports`) aggregates, in a single view, the indicators that support a staffing decision at organizational level rather than at the level of an individual project: the KPI summary (total and active projects, total resources, open positions, overall fill rate, fully-staffed projects), the needed-versus-assigned headcount per project, the demand-versus-supply comparison per skill, and the distribution of the resource pool by status and by stated availability (Chapter 4, UC RPT-1…4). As on the Availability screen, the data is refreshed by polling every 15 seconds, so the dashboard remains current without user intervention.

The aggregate indicators are computed in `ReportService`, and the part with real decision value — `computeSummary` — is again written as a pure function, tested separately for its edge cases (zero projects, zero requirements, over-allocation), as recorded in Table 3.2. **Figure 3.7** shows the dashboard with all four groups of indicators.

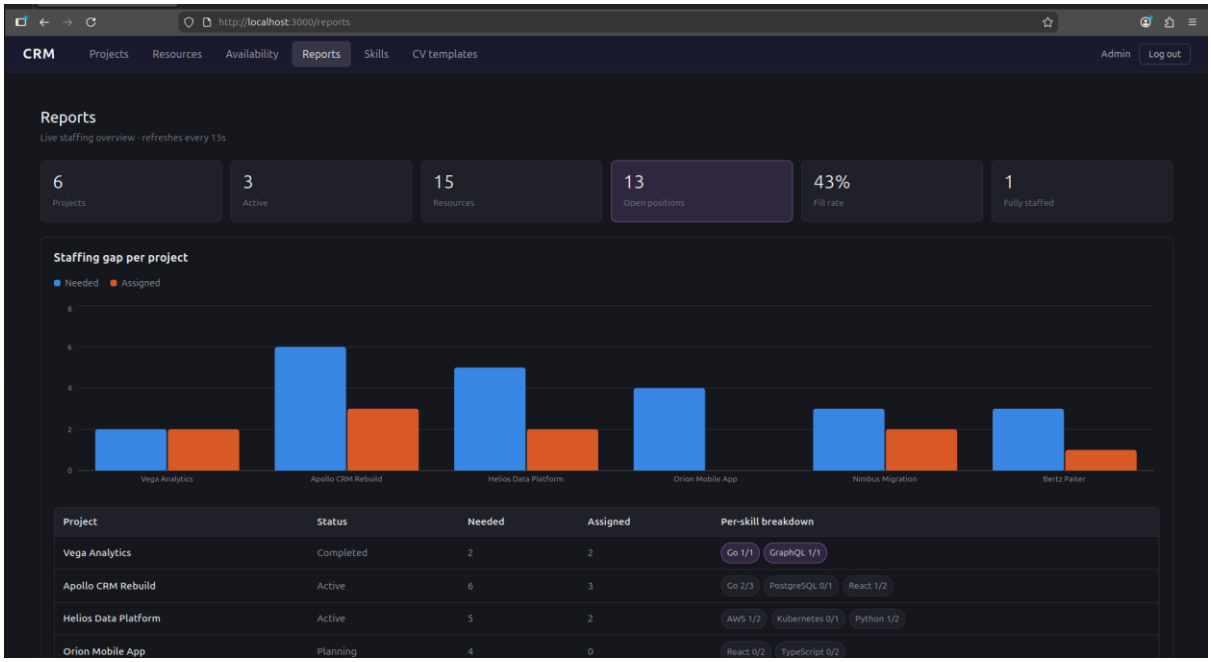

Figura 3.7. The Reports dashboard: KPIs, staffing gap, skill demand/supply and allocation.

3.3 Security

The security measures actually implemented are: password-based authentication with a `bcrypt` hash, a session with a signed JWT token and a fixed expiry (24 hours), authorization at GraphQL field level through the `@auth` directive, a single generic error message for failed authentication (without confirming the existence of an account), and explicit CORS restriction to the known origins of the development/production frontend (`http://localhost:5173`, `http://localhost:3000`), configured in `chi/go-chi/cors`.

A known limitation, discussed openly in section 5.2, is the absence of an explicit *query depth limiting* mechanism for GraphQL queries — the server runs with `gqlgen`'s default configuration, without a dedicated complexity plugin. Given that the application is an internal tool, with mandatory authentication on every field that exposes data, the practical risk is low, but it remains an item to harden before a public exposure of the service.

3.4 Containerization (Practical Recap)

Starting the complete system comes down to a single command (`docker compose up --build`), which brings up the database, automatically applies all migrations when the backend starts, and serves the static frontend through Nginx. Populating with demonstration data is strictly optional, through the separate `seed` profile (`docker compose --profile seed run --rm seed`), so as not to introduce test data into an environment that did not explicitly request it.

4 USE CASES

This chapter reproduces, as a functional reference document, the formal use-case specification of the system — 44 use cases, grouped into 9 functional modules, each described according to the standard template (Actors, Description, Preconditions, Trigger, Primary Flow, Alternative Flows, Exception Flows, Postconditions).

4.1 How to read this section

Each use case follows the same structure:

Field	Meaning
ID	Stable identifier, `<MODULE>-<N>`
Actors	Who initiates or participates in the use case
Description	What the use case accomplishes and why
Preconditions	System state required before the use case can begin
Trigger	The event that starts the use case
Primary Flow	The main successful path, as numbered steps
Alternative Flows	Valid variations on the primary path that still end in success
Exception Flows	Error and edge conditions, and how the system responds
Postconditions	Guaranteed system state after successful completion

Actor model: the system currently has a single actor role, **Authenticated User**. Every use case except Log In requires an active session; the system does not distinguish permission levels between accounts (the seeded `admin` and `user` accounts differ only in identity, not authority). Account creation is out of scope for this version — there is no self-registration use case; accounts exist only via direct provisioning (the database seed script).

4.2 Contents

7. Availability (Resource-Centric Matching)
8. Reports
9. Supporting Use Cases

Figure 4.1 gives an overview of the whole set: the actor, the nine functional modules and the cross-cutting supporting use cases that the module use cases include. **Figure 4.2** maps the same functionality onto the application's actual routes, showing where each group of use cases is reached from and how the authentication guard gates every route except `/login`. **Figure 4.3** shows the resource status state machine, which several use cases in section 4.3 and 4.6 modify.

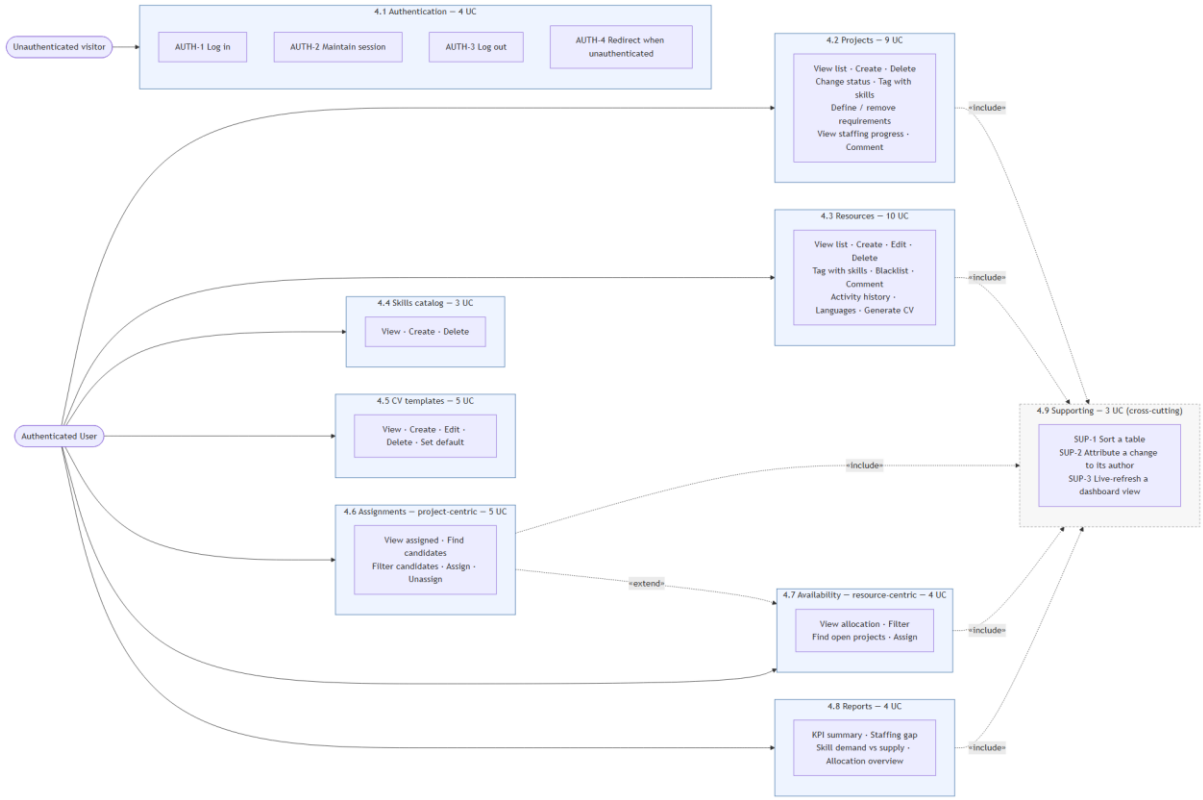

Figura 4.1. Use-case overview: the authenticated user and the nine functional modules.

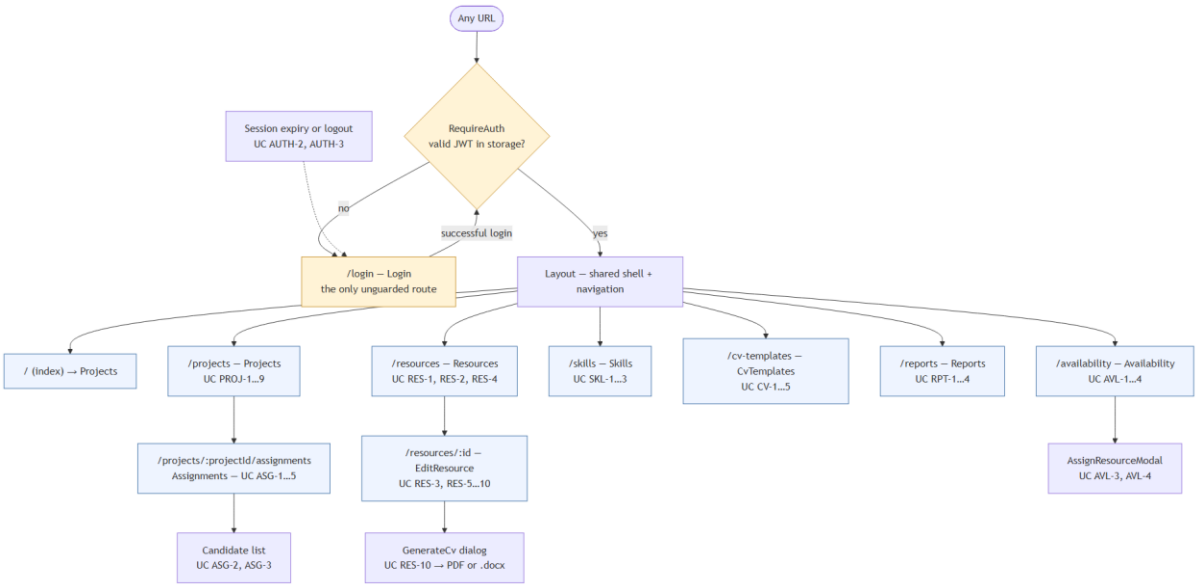

Figura 4.2. Navigation map of the application, with the authentication guard.

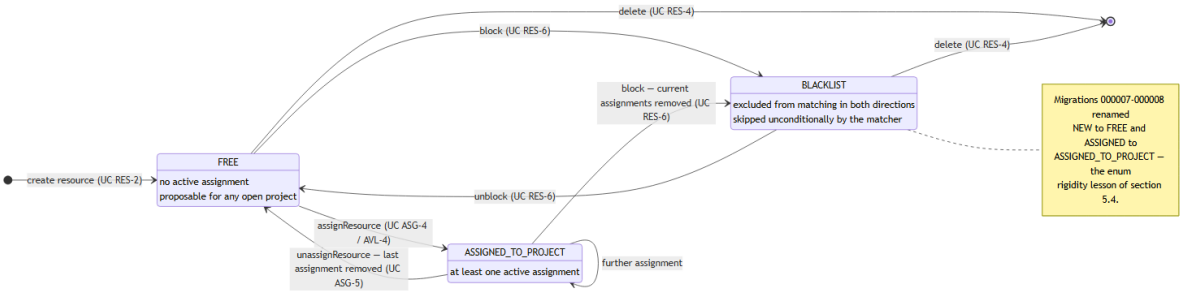

Figura 4.3. Resource status state machine.

4.3 Authentication

4.3.1 UC AUTH-1: Log In

Actors: Unauthenticated User

Description: Allows a user holding valid credentials to establish an authenticated session.

Preconditions: The user has a provisioned account (username, email, and password known to them). The user does not currently hold a valid session.

Trigger: The user opens the application while unauthenticated, or is redirected to `/login` after an unauthenticated or expired request.

Primary Flow:

1. System displays the login form with "Username or email" and "Password" fields.
2. User enters their username or email address.
3. User enters their password.
4. User submits the form.
5. System sends the credentials to the server.
6. Server locates the account by username or email and verifies the password against its stored hash.
7. Server issues a signed session token (valid 24 hours) and returns it together with the user's id, username, email, and display name.
8. System stores the session token and user profile in the browser.
9. System redirects the user to the Projects page.

Alternative Flows:

- **A1 — Already authenticated**: If the user navigates to `/login` while already holding a valid session, the system redirects immediately to the Projects page without displaying the form.
- **A2 — Login by email**: At step 2, the user may supply their email address instead of their username; the server matches either interchangeably.

Exception Flows:

- **E1 — Unknown account or wrong password**: At step 6, if no account matches or the password is incorrect, the server returns a single generic error ("invalid username/email or password"). The system displays this message inline and remains on the login form. No account-existence information is disclosed either way.

- **E2 — Incomplete form**: The submit control remains disabled until both fields are non-empty; the form cannot be submitted incomplete.
- **E3 — Request failure**: If the login request cannot be completed (network/server failure), the system shows a generic error and allows the user to retry.

Postconditions: On success, an authenticated session exists; every subsequent request to the server carries the session token.

4.3.2 UC AUTH-2: Maintain Session

Actors: Authenticated User

Description: Keeps the user signed in across page reloads and repeat visits without requiring re-authentication, for a bounded period.

Preconditions: The user has previously completed UC AUTH-1.

Trigger: The user reloads the application or returns to it in the same browser.

Primary Flow:

1. System reads the stored session token and user profile on startup.
2. System finds the token still within its 24-hour validity window.
3. System treats the user as authenticated and renders the requested page directly, without showing the login form.

Alternative Flows: None.

Exception Flows:

- **E1 — Expired token**: If the token has passed its 24-hour validity, the next request the user makes that requires authentication is rejected by the server; the system clears the stored session and redirects to `/login` (see UC AUTH-4).
- **E2 — No stored session**: If no token is present, the system treats the user as unauthenticated.

Postconditions: The user continues an existing session without re-entering credentials, or is routed to log in again if the session is no longer valid.

4.3.3 UC AUTH-3: Log Out

Actors: Authenticated User

Description: Ends the current session on demand.

Preconditions: The user holds an active session.

Trigger: The user selects "Log out" in the navigation bar, which displays their display name.

Primary Flow:

1. User selects "Log out".
2. System discards the stored session token and user profile.
3. System clears any cached application data tied to the session.
4. System redirects the user to `/login`.

Alternative Flows: None.

Exception Flows: None — logout is a local, client-side action and cannot fail against the server.

Postconditions: No valid session remains; the user must complete UC AUTH-1 to use the application again.

4.3.4 UC AUTH-4: Be Redirected to Login When Unauthenticated

Actors: Unauthenticated User (or a User whose session has just expired)

Description: Prevents any part of the application from being used without a valid session.

Preconditions: None.

Trigger: The user requests any route other than `/login` without a valid session, or a request made with an expired/invalid session token is rejected by the server.

Primary Flow:

1. System detects the absence of a valid session token before rendering a protected route.
2. System redirects the user to `/login`.

Alternative Flows:

- **A1 — Session expires mid-use**: If a session was valid when a page loaded but expires before a subsequent action, the server rejects that action with an authentication error; the system clears the stale session client-side and redirects to `/login` in response.

Exception Flows: None.

Postconditions: The user is presented with the login form (UC AUTH-1) and no protected data has been rendered.

4.4 Projects

4.4.1 UC PROJ-1: View Project List

Actors: Authenticated User

Description: Presents every project in the system as a sortable overview.

Preconditions: User is authenticated.

Trigger: User navigates to `/projects` (also the default landing route).

Primary Flow:

1. System requests the full project list from the server.
2. System displays each project's name, status, staffing progress, contact person, phone, email, who created it and when, and who last modified it and when.
3. System sorts the list by last-modified date, newest first, by default.

Alternative Flows:

- **A1 — Re-sort by another column**: User selects a different column header; system re-sorts the list by that field, descending. Selecting the same header again reverses the direction.
- **A2 — Expand a project's detail row**: User selects a project row; system reveals an inline panel with skill tags, staffing requirements, and comments (feeds UC PROJ-4 through PROJ-9).

Exception Flows:

- **E1 — Load failure**: If the project list cannot be retrieved, the system displays an inline error in place of the table.
- **E2 — No projects exist**: The table renders with a header row only.

Postconditions: The user has an up-to-date view of all projects.

4.4.2 UC PROJ-2: Create a Project

Actors: Authenticated User

Description: Registers a new project in the system.

Preconditions: User is authenticated and viewing the project list.

Trigger: User selects "+ Add new project".

Primary Flow:

1. System displays a form for name, contact person, phone, email, and status (defaulting to Planning).
2. User enters a name and a contact person (both required).
3. User optionally enters phone, email, and changes the status.
4. User confirms creation.
5. System creates the project, recording the current user as both creator and last modifier.
6. System closes the form and refreshes the project list to include the new project.

Alternative Flows:

- **A1 — Cancel**: User dismisses the form without submitting; no project is created.

Exception Flows:

- **E1 — Missing required fields**: The confirm control stays disabled until both name and contact person are non-empty; the form cannot be submitted without them.
- **E2 — Server rejects the request**: If creation fails server-side, the form remains open so the user can retry.

Postconditions: A new project exists with no skill tags, no staffing requirements, and no assignments, ready for UC PROJ-5 through PROJ-9.

4.4.3 UC PROJ-3: Delete a Project

Actors: Authenticated User

Description: Permanently removes a project and everything attached to it.

Preconditions: The target project exists.

Trigger: User selects "Delete" on a project row.

Primary Flow:

1. System sends the delete request immediately (no confirmation prompt is shown).
2. System removes the project along with its skill tags, staffing requirements, assignments, and comments.
3. System refreshes the project list; if the deleted project's detail row was expanded, it is collapsed.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The project remains in the list and an error is surfaced.

Postconditions: The project and all dependent records are irreversibly removed. Any resources that were assigned to it revert to Free status if they have no other active assignment.

4.4.4 UC PROJ-4: Change a Project's Status

Actors: Authenticated User

Description: Moves a project through its lifecycle.

Preconditions: Project detail row is expanded (UC PROJ-1, A2).

Trigger: User selects a different value in the project's status dropdown (Planning / Active / On Hold / Completed).

Primary Flow:

1. User selects a new status.
2. System updates the project, recording the current user as last modifier.
3. System reflects the new status in the project list and in any open reports/matching views.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the update**: The previous status is retained and an error is surfaced.

Postconditions: The project's status is updated everywhere it is referenced (staffing reports, matcher eligibility — a project moved to Completed is no longer offered as a match target in UC ASG-2 or UC AVL-3).

4.4.5 UC PROJ-5: Tag a Project with Skills

Actors: Authenticated User

Description: Records which skills a project involves, independent of specific headcount targets.

Preconditions: Project detail row is expanded; at least one skill exists in the catalog (UC SKL-2).

Trigger: User selects/deselects a skill chip in the project's detail panel.

Primary Flow:

1. User selects an untagged skill.
2. System attaches the skill to the project.
3. Detail panel reflects the skill as selected.

Alternative Flows:

- **A1 — Untag a skill**: User deselects an already-tagged skill; system removes the tag.

Exception Flows:

- **E1 — No skills defined**: The skill-tagging section is omitted from the detail panel until at least one skill exists in the catalog.
- **E2 — Server rejects the toggle**: The chip's state reverts and an error is surfaced.

Postconditions: The project's tagged-skill set is updated. These tags are used as the fallback matching signal (UC ASG-2, UC AVL-3) whenever the project has no explicit staffing requirements (UC PROJ-6).

4.4.6 UC PROJ-6: Define Staffing Requirements

Actors: Authenticated User

Description: States how many resources a project needs for a given skill — the basis for staffing-gap and fill-rate reporting.

Preconditions: Project detail row is expanded; at least one skill exists in the catalog.

Trigger: User selects a skill and a headcount in the "Needed resources" section and confirms.

Primary Flow:

1. User selects a skill not yet listed as a requirement.
2. User enters a needed count (a positive integer).
3. User confirms.
4. System records the requirement.
5. Detail panel lists the requirement with its current filled/needed count (initially 0/N).

Alternative Flows:

- **A1 — Change an existing requirement's count**: User edits the needed-count field directly on an existing requirement row; system updates it in place.

Exception Flows:

- **E1 — No skill selected**: The confirm control is disabled until a skill is chosen.
- **E2 — Invalid count**: A count below 1 is not accepted; the field will not commit a value outside the valid range.

- **E3 — All catalog skills already required**: The "add requirement" control is hidden once every catalog skill already has a requirement on this project.

Postconditions: The project's staffing requirement for that skill exists (or is updated), immediately reflected in UC PROJ-8, UC RPT-2, and the matcher scoring in UC ASG-2/UC AVL-3.

4.4.7 UC PROJ-7: Remove a Staffing Requirement

Actors: Authenticated User

Description: Withdraws a previously defined headcount target for a skill.

Preconditions: The project has at least one staffing requirement.

Trigger: User selects "Remove" on a requirement row.

Primary Flow:

1. System deletes the requirement.
2. Detail panel removes the row; the skill becomes available again for UC PROJ-6.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The requirement remains listed and an error is surfaced.

Postconditions: Existing assignments tagged against the removed requirement's skill are **not** deleted — they remain as assignments, but no longer count toward a specific open position for reporting purposes.

4.4.8 UC PROJ-8: View Staffing Progress

Actors: Authenticated User

Description: Shows how close a project is to being fully staffed, per skill and overall.

Preconditions: Project detail row is expanded, or the user is viewing Reports (UC RPT-2).

Trigger: Automatic — computed whenever the project's requirements or assignments are displayed.

Primary Flow:

1. For each staffing requirement, system displays filled/needed (e.g. "2/3 Go").
2. System visually distinguishes a requirement that is fully met (filled ≥ needed).
3. System displays the project's overall filled/needed total in the project list.

Alternative Flows: None — this is a read-only derived view.

Exception Flows: None.

Postconditions: None (informational use case).

4.4.9 UC PROJ-9: Comment on a Project

Actors: Authenticated User

Description: Maintains a free-text activity trail on a project.

Preconditions: Project detail row is expanded.

Trigger: User writes a comment and submits it (Enter, or the Save control).

Primary Flow:

1. User types comment text into the compose box.
2. User submits (presses Enter without Shift, or selects "Save").
3. System records the comment, attributed to the current user and timestamped.
4. System clears the compose box and appends the new comment to the list.

Alternative Flows:

- **A1 — Multi-line comment**: User inserts a line break with Shift+Enter before submitting; the comment retains the formatting.

Exception Flows:

- **E1 — Empty comment**: Submitting blank or whitespace-only text has no effect; nothing is recorded.
- **E2 — Server rejects the request**: The compose box retains the unsent text so the user can retry.

Postconditions: The comment is permanently attached to the project's history.

4.5 Resources

4.5.1 UC RES-1: View Resource List

Actors: Authenticated User

Description: Presents every resource in the system as a sortable overview.

Preconditions: User is authenticated.

Trigger: User navigates to `/resources`.

Primary Flow:

1. System requests the full resource list.
2. System displays each resource's name, birthdate, driving licence, car ownership, phone, availability, status, and last-modified date.
3. System sorts by last-modified date, newest first, by default.

Alternative Flows:

- **A1 — Re-sort by another column**: Same behavior as UC PROJ-1, A1.
- **A2 — Open a resource's full profile**: User selects a resource row; system navigates to `/resources/:id` (feeds UC RES-3 onward).

Exception Flows:

- **E1 — Load failure**: An inline error replaces the table.
- **E2 — No resources exist**: The table renders with a header row only.

Postconditions: The user has an up-to-date view of all resources.

4.5.2 UC RES-2: Create a Resource

Actors: Authenticated User

Description: Registers a new resource (person) in the system.

Preconditions: User is authenticated and viewing the resource list.

Trigger: User selects "+ Add new resource".

Primary Flow:

1. System displays a form for name, birthdate, phone, email, full address (street, number, block, flat, zip code, city, county, country), driving licence, car ownership, and availability.
2. User enters a name (required).
3. User optionally completes the remaining fields.
4. User confirms creation.
5. System creates the resource with status Free, recording the current user as creator and last modifier.
6. System closes the form and refreshes the resource list.

Alternative Flows:

- **A1 — Cancel**: User dismisses the form without submitting.

Exception Flows:

- **E1 — Missing name**: The confirm control stays disabled until a name is entered.
- **E2 — Server rejects the request**: The form remains open for retry.

Postconditions: A new resource exists with status Free, no skills, no languages, no activities, and no assignments.

4.5.3 UC RES-3: Edit a Resource's Details

Actors: Authenticated User

Description: Updates a resource's personal, contact, address, and availability information.

Preconditions: User has navigated to a resource's profile (`/resources/:id`).

Trigger: User modifies one or more fields and confirms.

Primary Flow:

1. System displays the resource's current details pre-filled.
2. User modifies any of name, birthdate, phone, email, address fields, driving licence, car, or availability.
3. User confirms.
4. System saves the changes, recording the current user as last modifier.
5. System reflects the updated details on the profile and in the resource list.

Alternative Flows: None.

Exception Flows:

- **E1 — Missing name**: The name field cannot be cleared and saved empty.
- **E2 — Server rejects the request**: Unsaved changes remain in the form for retry.

Postconditions: The resource's stored details match the submitted values; `modifiedBy`/`modifiedAt` are updated.

4.5.4 UC RES-4: Delete a Resource

Actors: Authenticated User

Description: Permanently removes a resource and everything attached to it.

Preconditions: The target resource exists.

Trigger: User selects "Delete" on a resource row.

Primary Flow:

1. System sends the delete request immediately (no confirmation prompt is shown).
2. System removes the resource along with its skill tags, activities, languages, comments, and assignments.
3. System refreshes the resource list.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The resource remains in the list and an error is surfaced.

Postconditions: The resource and all dependent records are irreversibly removed. Any projects it was assigned to lose that assignment.

4.5.5 UC RES-5: Tag a Resource with Skills

Actors: Authenticated User

Description: Records which skills a resource has — the primary signal used by both matchers (UC ASG-2, UC AVL-3).

Preconditions: User is on the resource's profile; at least one skill exists in the catalog.

Trigger: User selects/deselects a skill chip.

Primary Flow:

1. User selects an untagged skill.
2. System attaches the skill to the resource.
3. Profile reflects the skill as selected.

Alternative Flows:

- **A1 — Untag a skill**: User deselects an already-tagged skill; system removes the tag.

Exception Flows:

- **E1 — No skills defined**: The tagging section is omitted until the catalog has at least one skill.
- **E2 — Server rejects the toggle**: The chip's state reverts and an error is surfaced.

Postconditions: The resource's tagged-skill set is updated, immediately affecting its match score wherever it is evaluated as a candidate.

4.5.6 UC RES-6: Block (Blacklist) a Resource

Actors: Authenticated User

Description: Marks a resource as permanently ineligible for assignment, e.g. following a conduct issue.

Preconditions: The resource is not already blacklisted.

Trigger: User selects "Block" on a resource row.

Primary Flow:

1. System sends the block request immediately (no confirmation prompt is shown).
2. System removes every current project assignment the resource holds.
3. System sets the resource's status to Blacklist.
4. System refreshes the resource's displayed status everywhere it appears.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The resource's status is unchanged and an error is surfaced.

Postconditions: The resource holds no project assignments and is excluded from candidate matching (UC ASG-2) and from being offered a match (UC AVL-3) until unblocked at the data level — there is no "unblock" action in the current UI.

4.5.7 UC RES-7: Comment on a Resource

Actors: Authenticated User

Description: Maintains a free-text activity trail on a resource, identical in behavior to UC PROJ-9.

Preconditions: User is on the resource's profile.

Trigger: User writes a comment and submits it.

Primary Flow: Identical to UC PROJ-9, steps 1–4, applied to the resource's comment thread.

Alternative Flows: Same as UC PROJ-9, A1.

Exception Flows: Same as UC PROJ-9, E1–E2.

Postconditions: The comment is permanently attached to the resource's history.

4.5.8 UC RES-8: Track Work Activity / History

Actors: Authenticated User

Description: Records named periods of work experience for a resource (e.g. project onboarding, a prior role), forming the work-history timeline later used in CV generation (UC RES-10).

Preconditions: User is on the resource's profile.

Trigger: User selects "+ Add new activity", or edits/removes an existing one.

Primary Flow:

1. User enters an activity name (required) and an optional description.
2. User selects a start month and an end month via month/year dropdowns.
3. User leaves the field (blur) or navigates away, at which point the activity is saved automatically.
4. System persists the activity against the resource.

Alternative Flows:

- **A1 — Edit an existing activity**: User changes the name, description, or dates of an already-saved activity; the same auto-save behavior applies.
- **A2 — Remove an activity**: User selects "Delete" on an activity row; system removes it immediately.
- **A3 — Discard an unsaved blank row**: User adds a new activity row but leaves it entirely empty; it is not persisted.

Exception Flows:

- **E1 — End date before start date**: System displays "End date must not be before start date" beneath the offending row and does not save it until corrected.
- **E2 — Missing name**: A blank activity is not saved (see A3) — a row is only persisted once it has both a name and valid dates.
- **E3 — Server rejects the save**: An error is surfaced; the row's edits remain in the form for correction.

Postconditions: The resource's activity list reflects all saved entries, ordered by start date.

4.5.9 UC RES-9: Manage Languages and Proficiency

Actors: Authenticated User

Description: Records which languages a resource speaks and at what proficiency level.

Preconditions: User is on the resource's profile.

Trigger: User selects a language and a proficiency level and confirms, or adjusts/removes an existing entry.

Primary Flow:

1. User selects a language from the shared catalog (languages already assigned to this resource are not offered again).
2. User selects a CEFR proficiency level (A1–C2) or Native.
3. User confirms.
4. System attaches the language and level to the resource.
5. Resource profile lists the language with its level.

Alternative Flows:

- **A1 — Change an existing language's level**: User selects a different level in that language's row; system updates it immediately.
- **A2 — Add a language not yet in the catalog**: User selects "New language…", enters a name, selects a level, and confirms; system creates the catalog entry and attaches it to the resource in one action.
- **A3 — Remove a language**: User selects "Remove" on a language row; system detaches it from the resource (the catalog entry itself is unaffected).

Exception Flows:

- **E1 — No language selected**: The "Add" control is disabled until a language is chosen.
- **E2 — Blank new-language name**: The "Create & add" control is disabled until a name is entered.
- **E3 — Server rejects the request**: An inline error is shown; the attempted change is not reflected.

Postconditions: The resource's language set (with levels) is up to date and available to CV generation (UC RES-10).

4.5.10 UC RES-10: Generate a CV

Actors: Authenticated User

Description: Produces a downloadable CV document for a resource from a chosen template.

Preconditions: At least one CV template exists (UC CV-2).

Trigger: User selects "Generate CV" on a resource's profile.

Primary Flow:

1. System displays a dialog offering every CV template (the one marked default is pre-selected) and a file-type choice (PDF or Word).
2. User optionally selects a different template.
3. User selects a file type.
4. User confirms generation.
5. System renders the resource's details, skills, languages, and activities into the chosen template.
6. For PDF, system opens the browser's print dialog so the user can save it as PDF; for Word, system downloads an editable `.docx` file directly.
7. System closes the dialog.

Alternative Flows:

- **A1 — No default template set**: System pre-selects the first template in the list instead.

Exception Flows:

- **E1 — No templates exist**: The dialog shows "No CV templates exist yet" and generation cannot proceed until UC CV-2 is completed.
- **E2 — Generation fails server-side**: An inline error is shown and the dialog remains open for retry.

Postconditions: The user has an exported CV document; no data is modified. Comments, status, and project assignments are never included in the output, regardless of template.

4.6 4.4 Skills Catalog

4.6.1 UC SKL-1: View Skills

Actors: Authenticated User

Description: Lists every skill defined in the system.

Preconditions: User is authenticated.

Trigger: User navigates to `/skills`.

Primary Flow:

1. System requests and displays every skill with its optional description.

Alternative Flows: None.

Exception Flows:

- **E1 — No skills exist**: System displays "No skills defined yet."
- **E2 — Load failure**: An inline error is shown.

Postconditions: None (read-only).

4.6.2 UC SKL-2: Create a Skill

Actors: Authenticated User

Description: Adds a new skill to the shared catalog, making it available for tagging on both projects and resources.

Preconditions: User is viewing the skill list.

Trigger: User selects "+ Add new skill".

Primary Flow:

1. System displays a form for name (required) and an optional description.
2. User enters a name and, optionally, a description.
3. User confirms.
4. System creates the skill.
5. System closes the form and refreshes the skill list.

Alternative Flows:

- **A1 — Cancel**: User dismisses the form without submitting.

Exception Flows:

- **E1 — Missing name**: The confirm control stays disabled until a name is entered.
- **E2 — Duplicate name**: Skill names must be unique; the server rejects a duplicate and the form remains open with an error.

Postconditions: The skill is available system-wide for UC PROJ-5, PROJ-6, and RES-5.

4.6.3 UC SKL-3: Delete a Skill

Actors: Authenticated User

Description: Removes a skill from the catalog entirely.

Preconditions: The target skill exists.

Trigger: User selects "Remove" on a skill row.

Primary Flow:

1. System sends the delete request immediately (no confirmation prompt is shown).
2. System removes the skill and untags it from every project and resource that referenced it, and removes any staffing requirements built on it.
3. System refreshes the skill list.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The skill remains listed and an error is surfaced.

Postconditions: The skill no longer appears anywhere in the system, including historical staffing requirements.

4.7 CV Templates

4.7.1 UC CV-1: View CV Templates

Actors: Authenticated User

Description: Lists every CV template defined in the system.

Preconditions: User is authenticated.

Trigger: User navigates to `/cv-templates`.

Primary Flow:

1. System requests and displays every template's name, description, default marker (if applicable), and last-updated date.

Alternative Flows: None.

Exception Flows:

- **E1 — No templates exist**: System displays "No templates defined yet."
- **E2 — Load failure**: An inline error is shown.

Postconditions: None (read-only).

4.7.2 UC CV-2: Create a CV Template

Actors: Authenticated User

Description: Defines a new HTML-based layout that resource data can be rendered into.

Preconditions: User is viewing the template list.

Trigger: User selects "+ Add new template".

Primary Flow:

1. System opens the template editor.
2. User enters a name, an optional description, and the template's HTML content (using the supported placeholders for resource data).
3. User optionally marks the template as the default.
4. User confirms.
5. System validates the HTML structure.
6. System creates the template; if marked default, any previously default template is unmarked in the same operation.
7. System closes the editor and refreshes the template list.

Alternative Flows:

- **A1 — Cancel**: User dismisses the editor without submitting.

Exception Flows:

- **E1 — Invalid template HTML**: If the content fails structural validation, the server rejects the save and the editor remains open with the offending content intact for correction.
- **E2 — Server rejects the request**: The editor remains open for retry.

Postconditions: The template is available for UC RES-10. If marked default, it is the only default template in the system.

4.7.3 UC CV-3: Edit a CV Template

Actors: Authenticated User

Description: Modifies an existing template's name, description, or content.

Preconditions: The target template exists.

Trigger: User selects "Edit" on a template row.

Primary Flow:

1. System opens the editor pre-filled with the template's current values.
2. User modifies the name, description, and/or HTML content.
3. User confirms.
4. System validates and saves the changes.
5. System closes the editor and refreshes the template list.

Alternative Flows:

- **A1 — Cancel**: User dismisses the editor without saving; no changes are applied.

Exception Flows:

- **E1 — Invalid template HTML**: Same as UC CV-2, E1.
- **E2 — Server rejects the request**: The editor remains open with unsaved edits intact.

Postconditions: The template's content reflects the submitted changes; `modifiedAt` is updated.

4.7.4 UC CV-4: Delete a CV Template

Actors: Authenticated User

Description: Removes a template from the system.

Preconditions: The target template exists.

Trigger: User selects "Remove" on a template row.

Primary Flow:

1. System asks the user to confirm ("Delete the template "…"? This cannot be undone.").
2. User confirms.
3. System deletes the template.
4. System refreshes the template list.

Alternative Flows:

- **A1 — Decline the confirmation**: User cancels the prompt; the template is not deleted.

Exception Flows:

- **E1 — Server rejects the request**: The template remains listed and an error is surfaced.

Postconditions: The template is no longer available for UC RES-10. If it was the default template, no template is marked default until another is explicitly set (UC CV-5).

4.7.5 UC CV-5: Set the Default Template

Actors: Authenticated User

Description: Designates which template is pre-selected whenever a CV is generated.

Preconditions: The target template exists.

Trigger: User marks a template as default while creating (UC CV-2) or editing (UC CV-3) it.

Primary Flow:

1. User enables the "default" option on a template and saves.
2. System unmarks whichever template was previously default (if any) and marks the new one, atomically.
3. Template list shows the "Default" badge on the newly designated template only.

Alternative Flows: None.

Exception Flows:

- **E1 — Server rejects the request**: The default marker is unchanged.

Postconditions: Exactly one template — or none, if all defaults have been cleared and never reset — is marked default at any time.

4.8 Assignments (Project-Centric Matching)

4.8.1 UC ASG-1: View a Project's Assigned Resources

Actors: Authenticated User

Description: Shows who currently holds an assignment on a specific project.

Preconditions: The target project exists.

Trigger: User selects "Assignments" on a project row, navigating to `/projects/:projectId/assignments`.

Primary Flow:

1. System requests and displays every current assignment for the project — resource name, contact info, which staffing requirement (if any) it fills, and when it was assigned.
2. System displays the project's staffing-requirement progress strip above the list (see UC PROJ-8).

Alternative Flows: None.

Exception Flows:

- **E1 — Project not found**: System displays "Project #… not found."
- **E2 — No assignments yet**: System displays a prompt to assign someone from the candidate list (UC ASG-2).

Postconditions: None (read-only).

4.8.2 UC ASG-2: Find Candidate Resources for a Project

Actors: Authenticated User

Description: Ranks resources by how well they fit a project's remaining open positions, to support a staffing decision.

Preconditions: User is on a project's Assignments page.

Trigger: Automatic on page load, and re-evaluated whenever filters (UC ASG-3) change.

Primary Flow:

1. System determines the project's still-unfilled skill requirements (falling back to the project's tagged skills if no requirements are defined).
2. System scores every resource by how many of those unfilled skills it has.
3. System excludes blacklisted resources and resources already assigned to this project.
4. System ranks candidates by score (descending), then by soonest stated availability, then by name.
5. System displays each candidate with its score and which specific skills matched.

Alternative Flows:

- **A1 — Project has no requirements and no tagged skills**: Every eligible candidate scores 0 and is ranked by availability then name, rather than the list being empty.

Exception Flows:

- **E1 — No eligible candidates**: System displays "No free resources match the required skills for this project" (or the unfiltered equivalent).
- **E2 — Load failure**: An inline error is shown.

Postconditions: None (read-only, feeds UC ASG-4).

4.8.3 UC ASG-3: Filter Candidate Resources

Actors: Authenticated User

Description: Narrows the candidate list from UC ASG-2 to a more specific pool.

Preconditions: User is viewing the candidate list.

Trigger: User adjusts the search box, skill filter, availability filter, or the "unassigned only" toggle.

Primary Flow:

1. User enters a name search term, and/or selects a specific skill, and/or a specific availability, and/or enables "unassigned only".
2. System re-requests the ranked candidate list with those filters applied.
3. System displays the filtered, still-ranked results.

Alternative Flows:

- **A1 — Clear all filters**: User resets search to empty, skill/availability to "Any", and disables "unassigned only"; system returns to the unfiltered ranking of UC ASG-2.

Exception Flows:

- **E1 — No results under the current filters**: System displays "No resources match these filters."

Postconditions: None (read-only).

4.8.4 UC ASG-4: Assign a Resource to a Project

Actors: Authenticated User

Description: Places a candidate resource onto a project, optionally against a specific open staffing requirement.

Preconditions: A candidate is visible in UC ASG-2/ASG-3; the resource is not blacklisted.

Trigger: User selects "Assign" on a candidate row.

Primary Flow:

1. System pre-selects the "assign as" skill slot to the top unfilled requirement the candidate matches, if any (otherwise "General").
2. User optionally changes the target skill slot.
3. User selects "Assign".
4. System records the assignment against the selected skill (or as a general, untagged assignment).
5. System sets the resource's status to "Assigned to project".

6. System moves the resource from the candidate list to the assigned list, and updates the project's staffing progress (UC PROJ-8) accordingly.

Alternative Flows:

- **A1 — Assign without a specific skill**: User leaves the slot as "General"; the assignment counts toward the project's overall headcount but not toward any specific requirement's fill count.
- **A2 — Assign a resource already on other projects**: Permitted — a resource may hold assignments on multiple projects simultaneously.
- **A3 — Re-assign to a different skill**: Assigning a resource already on this project again (e.g. from the Availability screen) updates which skill slot it fills rather than creating a duplicate assignment.

Exception Flows:

- **E1 — Resource is blacklisted**: The server refuses the assignment ("cannot assign a blacklisted resource"); the UI does not offer this action for blacklisted resources in the first place.
- **E2 — Server rejects the request**: An error is surfaced and the resource remains unassigned.

Postconditions: The assignment exists; the resource's status and the project's staffing figures are updated everywhere they are displayed (Assignments, Availability, Reports).

4.8.5 UC ASG-5: Unassign a Resource from a Project

Actors: Authenticated User

Description: Removes a resource's assignment to a project.

Preconditions: The resource currently holds an assignment on this project.

Trigger: User selects "Remove" on an assigned-resource row.

Primary Flow:

1. System deletes the assignment.
2. System checks whether the resource holds any other active assignment.
3. If none remain, system sets the resource's status back to Free.
4. System moves the resource from the assigned list back into the candidate pool, and updates the project's staffing progress.

Alternative Flows:

- **A1 — Resource retains other assignments**: If the resource is assigned to other projects, its status remains "Assigned to project" rather than reverting to Free.

Exception Flows:

- **E1 — Resource is blacklisted**: Unassigning never overrides a blacklist status back to Free, even if it was the resource's last assignment.
- **E2 — Server rejects the request**: The assignment remains and an error is surfaced.

Postconditions: The assignment no longer exists; the resource's status reflects its remaining assignments (if any).

4.9 Availability (Resource-Centric Matching)

4.9.1 UC AVL-1: View Resource Availability and Allocation

Actors: Authenticated User

Description: Provides a resource-first overview of who is free, who is loaded, and how soon each person can start — the operational counterpart to UC ASG-1.

Preconditions: User is authenticated.

Trigger: User navigates to `/availability`.

Primary Flow:

1. System requests and displays every resource with its status, stated availability, current project count, the names of those projects, and contact info.
2. System automatically refreshes this view every 15 seconds without user action.

Alternative Flows: None.

Exception Flows:

- **E1 — Load failure**: An inline error is shown.
- **E2 — No resources exist**: The table renders empty.

Postconditions: None (read-only).

4.9.2 UC AVL-2: Filter the Availability List

Actors: Authenticated User

Description: Narrows the availability overview to a specific subset of resources.

Preconditions: User is viewing the availability list.

Trigger: User adjusts the search box, availability filter, skill filter, or the "unassigned only" toggle.

Primary Flow:

1. User enters a name search term, and/or selects a specific availability, and/or a specific skill, and/or enables "unassigned only".
2. System filters the displayed list client-side against the current criteria.
3. System shows a count of how many resources match out of the total.

Alternative Flows:

- **A1 — Clear all filters**: System returns to showing every resource.

Exception Flows:

- **E1 — No results under the current filters**: System displays "No resources match these filters."

Postconditions: None (read-only).

4.9.3 UC AVL-3: Find Open Projects for a Resource

Actors: Authenticated User

Description: The mirror image of UC ASG-2 — starting from a specific resource, ranks which open projects it would be a good fit for.

Preconditions: The target resource is not blacklisted.

Trigger: User selects "Assign" on a resource row in the Availability list.

Primary Flow:

1. System opens a dialog scoped to the selected resource.
2. System excludes projects with status Completed and projects the resource is already assigned to.
3. System determines each remaining project's still-unfilled skill requirements.
4. System scores the resource against those unfilled skills per project.
5. System excludes projects with no unfilled requirements at all (i.e. shows only "open" projects), unless the underlying query is explicitly asked to include fully-staffed ones.
6. System ranks the remaining projects by match score (descending), then by how many open positions the project has (descending), then by project name.
7. System displays each candidate project with its match score, status, and open-position count.

Alternative Flows:

- **A1 — No skill overlap with any open project**: Every open, eligible project is still listed, scored 0, ranked by open-position count then name — the resource is not hidden from projects it simply doesn't have a specific skill match for.

Exception Flows:

- **E1 — Resource is blacklisted**: The system yields no candidate projects at all for this resource (defense in depth; the UI additionally does not offer the "Assign" action for a blacklisted resource in the first place).
- **E2 — No open projects match**: Dialog displays "No open projects match right now."
- **E3 — Load failure**: An inline error is shown within the dialog.

Postconditions: None (read-only, feeds UC AVL-4).

4.9.4 UC AVL-4: Assign a Resource to a Project from Availability

Actors: Authenticated User

Description: Completes an assignment without leaving the Availability screen or navigating through Projects — functionally the same outcome as UC ASG-4, reached from the opposite direction.

Preconditions: The candidate-project dialog (UC AVL-3) is open.

Trigger: User selects "Assign" on a candidate project row within the dialog.

Primary Flow:

1. System pre-selects the "assign as" skill slot to the top unfilled requirement the resource matches, if any (otherwise "General").
2. User optionally changes the target skill slot.
3. User selects "Assign".
4. System records the assignment exactly as in UC ASG-4, steps 4–6.
5. System refreshes the Availability list, the candidate-project dialog, and the affected project's staffing figures.

Alternative Flows:

- **A1 — Assign without a specific skill**: Same as UC ASG-4, A1.

Exception Flows:

- **E1 — Server rejects the request**: Same as UC ASG-4, E2.

Postconditions: Identical to UC ASG-4 — the assignment exists and every dependent view (Assignments, Availability, Reports) reflects it.

4.10 Reports

4.10.1 UC RPT-1: View KPI Summary

Actors: Authenticated User

Description: Presents headline organizational metrics at a glance.

Preconditions: User is authenticated.

Trigger: User navigates to `/reports`.

Primary Flow:

1. System computes and displays: total projects, active projects, total resources, total open positions, overall fill rate (%), and count of fully-staffed projects.
2. System refreshes these figures automatically every 15 seconds.

Alternative Flows: None.

Exception Flows:

- **E1 — Load failure**: An inline error is shown in place of the dashboard.

Postconditions: None (read-only).

4.10.2 UC RPT-2: View Staffing Gap per Project

Actors: Authenticated User

Description: Compares needed vs. assigned headcount across all projects, to identify where staffing is falling short.

Preconditions: User is viewing Reports.

Trigger: Automatic, as part of UC RPT-1's page load.

Primary Flow:

1. System displays a bar chart of needed vs. assigned headcount per project.
2. System displays a supporting table with the full per-skill requirement breakdown for each project.

Alternative Flows: None.

Exception Flows:

- **E1 — No projects exist**: System displays "No projects yet" in place of the chart.

Postconditions: None (read-only).

4.10.3 UC RPT-3: View Skill Demand vs. Supply

Actors: Authenticated User

Description: Compares how many people each skill is requested for, system-wide, against how many resources actually have it — surfacing skill shortages.

Preconditions: User is viewing Reports.

Trigger: Automatic, as part of UC RPT-1's page load.

Primary Flow:

1. System aggregates, per skill, the total requested headcount across all projects (demand) and the count of non-blacklisted resources holding that skill (supply).
2. System displays this as a grouped bar chart, limited to skills with nonzero demand or supply.

Alternative Flows: None.

Exception Flows:

- **E1 — No skill requirements set anywhere**: System displays "No skill requirements set yet."

Postconditions: None (read-only).

4.10.4 UC RPT-4: View Resource Allocation Overview

Actors: Authenticated User

Description: Shows how the overall resource pool is split by status and by stated availability.

Preconditions: User is viewing Reports.

Trigger: Automatic, as part of UC RPT-1's page load.

Primary Flow:

1. System displays a donut chart of Free / Assigned / Blacklisted resource counts.
2. System displays a bar chart of the non-blacklisted pool bucketed by stated availability (ASAP / 1 / 2 / 3 weeks / not set).

Alternative Flows: None.

Exception Flows:

- **E1 — No resources exist**: System displays "No resources yet" in place of both charts.

Postconditions: None (read-only).

4.11 4.9 Supporting Use Cases

These use cases are not reached from a dedicated screen; they describe system behavior that participates in multiple use cases above.

4.11.1 UC SUP-1: Sort a Table

Actors: Authenticated User

Description: Reorders the Projects or Resources list by any column, described once here since it applies identically in UC PROJ-1 and UC RES-1.

Preconditions: User is viewing a sortable list.

Trigger: User selects a column header.

Primary Flow:

1. User selects a column header not currently used for sorting.
2. System re-sorts the list by that column, descending, and marks the header as active.

Alternative Flows:

- **A1 — Reverse the current sort**: User selects the already-active column header again; system flips the sort to ascending.
- **A2 — Switch to a different column**: User selects a different header while a sort is active; system switches to that column, defaulting to descending again.

Exception Flows: None — sorting is a purely local, client-side operation and cannot fail.

Postconditions: The list's display order reflects the selected column and direction until the user changes it again or leaves the page.

4.11.2 UC SUP-2: Attribute a Change to Its Author

Actors: System (invoked automatically by UC PROJ-2, PROJ-3, PROJ-4, RES-2, RES-3, RES-4, RES-6, PROJ-9, RES-7)

Description: Every creation, modification, or comment is stamped with the identity of whoever was authenticated at the time, rather than a fixed system account.

Preconditions: A create/update/comment use case is in progress under an authenticated session.

Trigger: Any of the invoking use cases reaching its persistence step.

Primary Flow:

1. System reads the current user's display name from the active session.
2. System records that name as the record's creator (on creation) or last modifier (on update), or as a comment's author.

Alternative Flows: None.

Exception Flows:

- **E1 — No user on the session (should not occur)**: Every write operation requires authentication, so this path is unreachable in normal operation; the system falls back to recording "system" rather than leaving the field blank, since the field cannot be empty.

Postconditions: The record's authorship is traceable to a real user.

4.11.3 UC SUP-3: Live-Refresh a Dashboard View

Actors: System (invoked automatically within UC AVL-1 and UC RPT-1–4)

Description: Keeps the Availability and Reports screens current without requiring a manual reload, since they summarize data that can change from other users' actions elsewhere in the system.

Preconditions: User is viewing Availability or Reports.

Trigger: A fixed 15-second interval elapses while the page remains open.

Primary Flow:

1. System silently re-requests the underlying data.
2. System updates the displayed figures/charts/table in place, without disrupting any in-progress user interaction (e.g. an open filter or dialog).

Alternative Flows: None.

Exception Flows:

- **E1 — Refresh request fails**: The previously displayed data remains on screen; the system retries on the next interval rather than surfacing a disruptive error for a background refresh.

Postconditions: The displayed data is no more than 15 seconds stale at any point while the page is open.

5 CONCLUSIONS

5.1 Results Obtained

The project produced a fully functional system that covers, as set out in the objectives of section 1.1, all three basic needs: project tracking, human resource tracking, and staff allocation assisted by a matching score. The central point of the personal contribution is the symmetric matching algorithm described in section 3.1.1 — implemented only once, as testable pure functions, and reused unchanged for both search directions (project → resources and resource → projects), thereby avoiding the divergence that maintaining two separate implementations would have introduced.

Tabel 5.1. The mapping between the initial requirements and the delivered features.

Requirement (section 2.1)	**Status**
Authentication and session with expiry	Delivered — JWT, 24h, single error message on failed authentication
Full project/resource management (CRUD, comments, status)	Delivered
Shared skill catalog	Delivered
CV templates with validation and a single default template	Delivered, with a transactional guarantee
Bidirectional project ↔ resource matching	Delivered, a single algorithm reused in both directions
Aggregate reports with automatic refresh	Delivered — KPIs, skill demand/supply, resource allocation
Startup with a single command (containerization)	Delivered — `docker compose up`, automatic migrations
Testability of the business logic without a database	Delivered — table-driven tests for the score, reports, CV validation
Depth/complexity limiting of GraphQL queries	Not implemented — see limitations (5.2)

5.2 Limitations

The system, as implemented at the time of writing this thesis, has a few known limitations, openly acknowledged:

- **A single authorization level.** All authenticated accounts have the same rights; there is no administrator/regular-user distinction, and creating new accounts is not possible from the interface (accounts exist only through direct provisioning in the database). For an internal tool with a small number of trusted users, this trade-off was acceptable; it would not be sufficient for a multi-organization exposure.

- **No GraphQL query complexity limiting.** The server runs with `gqlgen`'s default configuration, without a dedicated plugin for limiting the depth or complexity of a query — a theoretical API-abuse risk, reduced in practice by the authentication requirement on every field, but not explicitly addressed.
- **No soft delete.** Deleting a project, a resource or a skill is immediate and permanent; there is no recovery mechanism or intermediate "recycle bin".
- **Refresh by periodic polling, not by real-time subscriptions.** The Availability and Reports screens update every 15 seconds, not instantly on change — a simple solution, sufficient for the application's current scale, but one that would become inefficient with a large number of simultaneously connected clients.
- **No dedicated mobile version.** The interface is built for desktop use; it has not been explicitly optimized for small screens.

5.3 Future Development Directions

Starting from the limitations above, the natural directions for continuing the project are:

1. **Roles and permissions** — introducing a granular authorization level (for example, administrator vs. member), with minimal impact on the current schema, given that the `@auth` directive is already the central point through which every authorization check passes.
2. **GraphQL complexity limiting** — adding a `complexity limit`/`depth limit` plugin at the level of the `gqlgen` server, directly addressing the limitation identified in section 5.2.
3. **GraphQL subscriptions** — replacing the periodic refresh on the Availability and Reports screens with server-pushed updates (`graphql-ws` or equivalent), for genuinely instantaneous data.
4. **A complete audit history** — extending the current author traceability (UC SUP-2) with a full event log (who changed what, when, from which value to which value), useful both for auditing and for possible "undo" features.
5. **Notifications** — proactively alerting a user when a project they manage remains below its staffing requirements for an extended period, or when a resource with ASAP availability has no assignment.

5.4 Lessons Learned

Developing this project confirmed, in practice, a few design principles whose value is easy to underestimate in theory:

- **A single typed contract (the GraphQL schema) prevents entire classes of errors** which, in a REST architecture without an explicit contract, would have appeared as silent discrepancies between the data shape expected by the client and the one actually sent by the server.
- **Separating business logic from data access, in the form of pure functions**, is what made possible a fast, infrastructure-free test suite for exactly the part of the system with the highest decision value (the matching algorithm).
- **A constraint enforced at database level as well** (the partial unique index for the default CV template), not only at application level, proved necessary as a last line of defence against an inconsistent state, regardless of any future errors in the application code.
- **Choosing `TEXT` + `CHECK` instead of a native `ENUM`**, wherever a field has a real chance of evolving, avoided a class of costly migrations — a lesson learned directly from the correction later applied to a resource's status (migrations 000007–000008).

6 BIBLIOGRAPHY

[1] 99designs, *gqlgen — a GraphQL server library for Go*. [Online]. Available: https://gqlgen.com/

[2] The Apollo GraphQL team, *Apollo Client Documentation*. [Online]. Available: https://www.apollographql.com/docs/react/

[3] GraphQL Foundation, *GraphQL Specification*. [Online]. Available: https://spec.graphql.org/

[4] PostgreSQL Global Development Group, *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/

[5] The Go Authors, *The Go Programming Language Documentation*. [Online]. Available: https://go.dev/doc/

[6] golang-migrate, *Database migrations written in Go*. [Online]. Available: https://github.com/golang-migrate/migrate

[7] jmoiron, *sqlx — general purpose extensions to Go's database/sql*. [Online]. Available: https://github.com/jmoiron/sqlx

[8] React core team, *React Documentation*. [Online]. Available: https://react.dev/

[9] Docker Inc., *Docker Compose Documentation*. [Online]. Available: https://docs.docker.com/compose/

[10] golang-jwt, *JSON Web Tokens for Go*. [Online]. Available: https://github.com/golang-jwt/jwt

7 APPENDICES

7.1 Source Code

The complete source code of the application (Go backend, React frontend, GraphQL schema, SQL migrations, Docker configuration) is organized into two main directories at the root of the project, `backend/` and `frontend/`, according to the structure described in Chapter 2 and Chapter 3.

7.2 Project Website

https://github.com/SPHNVC/ADVSOFTFORHUMANRESOURCES/